



Dipartimento di Matematica e Informatica

Università degli Studi di Ferrara
Corso di Studi di Informatica

Tecnologie Server e Client Side, Human-Computer Interaction e
moderne applicazioni Web

a cura di:

Giacomo **Piva**, Federico **Bonini**, Ilaria **Conforto**

1. VCS, definizione e caratteristiche

UNITÀ 0

INTRODUZIONE

L1: Storia e cenni introduttivi

1

INDICE

1. Cenni storici
2. Gli Ipertesti
3. Il Web
4. Il Web 2.0

5. Gli identificatori

6. Messaggio di richiesta: HTTP

TECNOLOGIE SERVER SIDE

L1: HTTP

1. Cos'è HTTP?
2. Storia del HTTP
3. Funzionalità del HTTP 2.0
4. Tipi di trasmissione HTTP

L2: Web Server e modello Client - Server

1. Cos'è il modello Client-Server? 2. Come funziona?
3. Protocolli di comunicazione 4. Modello Pull e Push

L3: Git

L4: HTML

1. Cos'è?

L5: CSS

1. Cos'è?
2. Integrare codice CSS
3. Priorità tra gli stili

L6: OOPHP

1. Cos'è PHP?
2. Le Classi
3. I Metodi
4. Modificatori
 - Request
7. Messaggio di risposta: HTTP
 - Response
8. Limiti del HTTP
9. Protocollo IP
10. W3C
5. Stato
6. Modello iterativo e concorrente 7. Tecnologie Server Side
8. Pagine statiche e dinamiche 9. Subdirectory
10. Servizi di Hosting
2. Git
3. Bitbucket
4. JSP
2. I Tag
3. Web Semantico

4. Struttura di un foglio CSS
5. Gerarchia di codice
6. Margini e Padding
7. Bootstrap

5. Object Oriented Programming 6. PHP e HTML mixing
7. PHP 7

- 8. PHP 8
- 9. Le Funzioni Anonime

2

UNITÀ 2

HUMAN-COMPUTER INTERACTION

L1: HCI

- 1. Il fattore umano
- 2. Il modello RGB
- 3. RGB in CSS

L2: PRINCIPI DI UI DESIGN

- 1. Cos'è?
- 2. UX vs Design
- 3. Chiarezza
- 4. Consentire l'interazione
- 5. Conservare l'attenzione
- 6. Lasciare il controllo agli utenti
- 7. Favorire la manipolazione diretta

L3: INTERACTION DESIGN

- 1. Cos'è?
- 2. 1D - Parole
- 3. 2D - Rappresentazione Visiva

UNITÀ 3

TECNOLOGIE CLIENT SIDE

L1: JAVASCRIPT

- 1. Cos'è?
- 2. Browser
- 3. Le basi del linguaggio

L2: XML

- 1. Cos'è?
- 2. XSD, XML Schema
- 3. Namespace

- 4. Il modello HEX
- 5. I colori e il loro significato
- 6. La letteratura e Yabus
- 7. Hard e Soft Controls
- 8. Relazioni Controller-Display
- 9. Il Modello Concettuale
- 10. La Modalità
- 11. Errori di interazione

- 8. Azioni primarie e secondarie
- 9. Agevolare il prossimo passo
- 10. L'aspetto segue il comportamento
- 11. Consistenza
- 12. Forte gerarchia
- 13. Organizzazione intelligente
- 14. Mostrare progressivamente
- 15. Il momento 0

- 4. 3D - Spazio fisico
- 5. 4D - Tempo
- 6. 5D - Comportamento
- 7. Strategia e prototipazione

- 4. DOM
- 5. Eventi
- 6. Framework JavaScript
- 7. jQuery e le sue funzioni
- 8. Funzione \$
- 9. Binding
- 10. .each()
- 11. Animazioni

- 4. Ajax e Ajax con jQuery
- 5. XMLHttpRequest
- 6. JSON
- 7. API Rest

UNITÀ 4

MODERNE APPLICAZIONI WEB

L1: LARAVEL

1. Cos'è?
2. MVC
3. REST
4. Web Framework
5. URL e ROTTE
6. ORM
7. .ENV

Tim Berners-Lee



In questo capitolo introdurremo dal punto di vista storico la nascita della rete internet, partendo dalla creazione di Arpanet per poi finire con l'avvento dei siti più importanti che usiamo tutt'ora. Verranno inoltre introdotte le principali differenze tra l'utilizzo del FTP e del WWW, osservando le conseguenze che queste tecnologie hanno avuto sul web e sulla nascita del web 2.0. Si vedranno inoltre le principali caratteristiche di questi ultimi.



Prima



Adesso

5

1.1 CENNI STORICI

→ **1945:** Dr. Vannevar Bush (foto) propone a Roosevelt la macchina MEMEX, che raccoglie informazioni testuali e grafiche con la possibilità di definire collegamenti arbitrari tra di loro

→ **1957:** ARPA (Advanced Research Projects Agency), permette agli USA di concorrere con la Russia per la conquista dello spazio,



dopodichè utilizzata per scopi militari

- **1961**: Leonard Kleinrock pubblica il primo articolo sulla teoria delle reti a commutazione di pacchetto
- **1965**: grazie ad ARPANET un computer al MIT viene interconnesso con uno in California, inoltre Theodor Holm Nelson conia i termini “hypertext” e “hypermedia”
- **1967**: alla Brown University venne sviluppato il primo sistema basato su ipertesti, utilizzato nel programma Apollo
- **1969**: su ARPANET vengono interconnessi 4 nodi su linee a 50Kbps
- **1971**: Ray Tomlinson scrive il primo programma di email per inviare messaggi su rete distribuita
- **1972**: RFC (Request For Comments) per Telnet
- **1973**: RFC per FTP (File Transfer Protocol)
- **1977**: RFC per e-mail
- **1978**: Andrew Lippman all’MIT Architecture Machine Group guida un gruppo di ricercatori allo sviluppo del primo hypermedia system Aspen Movie Map
- **1980**: ARPANET (1000 nodi) bloccato per il primo virus informatico 6
- **1982**: ARPANET cambia nome e viene coniato il termine Internet → **1984**:
Internet in Italia
- **1987**: Internet conta 10000 nodi collegati
- **1989**: Internet conta 100000 nodi collegati e Tim Berners Lee propone il primo progetto per la gestione documentale
- **1990**: primo accesso a Internet in dial-up e viene utilizzato il termine World Wide Web per descrivere il progetto di Berners Lee del 1989
- **1991**: utilizzo commerciale della rete e nasce HTML, HTTP e URL
- **1993**: Marc Andreessen rilascia Mosaic-for-X su Unix, in seguito su Mac e Windows; inoltre fonda la Mosaic Communication Corp (futura Netscape) con Eric Bina

→ **1994**: Tim Berners Lee fonda il W3C

→ **1995**: Sun Microsystems rilascia Java e la navigazione www super il trasferimento dati (FTP) come volumi di traffico

→ **1998**: Google

→ **2000**: Napster e diffusione di virus



→ **2002**: Blog

→ **2004**: Tim O'Reilly introduce il concetto di Web 2.0 e di servizi di seconda generazione come: Social Network (Facebook nasce proprio nel 2004), Wiki e Blog

→ **Oggi**: nascita dello streaming e applicazioni come Spotify, Netflix e WhatsApp



7

1.2 GLI IPERTESTI

L'**ipertesto** è una forma di scrittura non lineare che si sviluppa attraverso una rete di nodi e collegamenti (link), permettendo al lettore di navigare liberamente tra i contenuti. Questo modello di fruizione rompe la sequenzialità tradizionale del testo scritto, offrendo percorsi multipli e personalizzabili. Ogni nodo può contenere testo, immagini, video o altri media, arricchendo l'esperienza di lettura e coinvolgendo diversi sensi.

1.3 IL WEB

YAHOO!

YAHOO!

Nasce nel gennaio 1994 da David Filo e Jerry Yang, un **portale di accesso al web** con lo scopo di essere il punto di riferimento per gli utenti per esplorarlo. Fornisce servizi gratuiti come: motore di ricerca, email



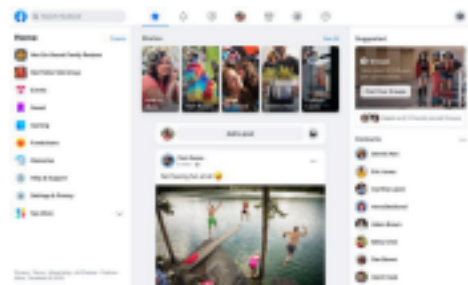
(da cui poter accedere ovunque), notizie e informazioni commerciali.

- tecnologie utilizzate:

- infrastruttura → Server FreeBSD (Unix)
- tecnologie server-side → Apache, PHP, Perl
- tecnologie client-server → JavaScript
- tecnologie di architettura → C, C++



Nasce il 4 febbraio 2004 da Mark Zuckerberg e co.; **social network** con lo scopo di collegare persone/aziende che hanno relazioni. Offre servizi come: ricerca (interna al sito web/app), condivisione di contenuti (video, foto, testo), gruppi di discussione (commenti, Messenger) e inserzioni pubblicitarie.



8

- tecnologie utilizzate:

- infrastruttura → Database Cassandra (NoSQL)
- tecnologia di base → HTML5
- tecnologia server-side → PHP
- tecnologia client-server → JavaScript
- tecnologia di architettura → C, C++, Memcached

in canali video. Offre la possibilità di: pubblicare video, cercare e streaming video.

YOUTUBE

Nasce il 14 febbraio 2005 da Steve Chan e co., **contenitore multimediale** che permette agli utenti di pubblicare filmati e di organizzarli

- tecnologie utilizzate:

- infrastruttura → Database MySQL
- tecnologia di base → HTML5



- tecnologia server-side → PHP, Perl, Python, Ruby
- tecnologia client-server → JavaScript

GOOGLE

Nasce il 4 settembre 1998 da Larry Page e Sergey Brin, il **motore di ricerca** più utilizzato al mondo; dal 2008 ha aggiunto altri servizi come Gmail, Maps e Docs, per poi ampliarsi con: Play, Now, Drive, Chrome, Translate, Earth, Street View, Meet, Chat, Hangouts, Classroom, ...

- tecnologie utilizzate:

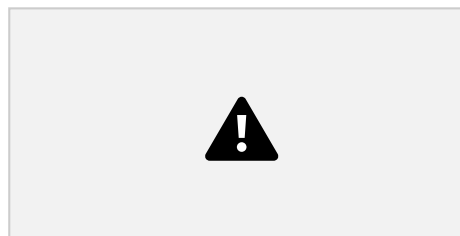
- infrastruttura → Cluster con 1000+ unità



- tecnologia server-side → Google Web

Server

- tecnologia client-server → JavaScript,
AJAX



- tecnologia di architettura → C, C++

Tra le tecnologie di infrastruttura utilizzate si ha **Google Cluster**, un insieme di computer/macchine connesse tra loro tramite una rete telematica che lavorano in parallelo. È una scelta economicamente efficiente in quanto più economica rispetto a super computer e i guasti hanno un effetto di degrado temporaneo del throughput.

Il software viene rappresentato da **PageRank™**: sistema che permette di classificare le pagine web in ordine di importanza relativamente ad una o più parole chiave esistenti all'interno del sito.

Google è suddiviso in 3 parti principali:

1. **Googlebot**: legge le pagine
2. **Indexer**: indicizza le pagine
3. **Query Processor**: estrae le pagine

GOOGLEBOT

È lo spider web di Google; partendo da una pagina web ne scarica il contenuto passandolo all'indexer, dopodiché segue i link presenti nella pagina passando così da un sito all'altro. Ha la funzione di: cercare e indicizzare i nuovi siti, aggiornare i siti esistenti, evitare inserimenti automatici ("non sono un robot") e riconoscere e scartare indirizzi di spam.

Durante il fetch delle pagine, Googlebot, **compone una lista di link** da visitare, ne **gestisce i duplicati** della lista e **calcola quando rivisitare una pagina** (in media il tempo standard era di 1 mese, comunque chi sviluppa il sito può specificare entro quanto ritornare a visitarlo).

10

GOOGLE INDEXER

Quando riceve il contenuto delle pagine dallo spider, **ricava i termini** di ricerca all'interno della pagina; ad ogni termine vengono **associati i documenti** che lo contengono e ad ogni documento viene **associato un Ranking** in relazione ad ogni termine che contiene. Quando ricava i termini vengono eliminate le stop words (come articoli grammaticali), riscritto in minuscolo e fatte altre operazioni di filtraggio.

Google ha una lista di termini (duplicati in svariate lingue) con ognuno il suo punteggio; inoltre prende più in considerazione:

1. URL
2. H1

3. META DESCRIPTION e META TITLE

4. H*

5. PARAGRAFI

GOOGLE QUERY PROCESSOR

Interpreta le query degli utenti, ovvero le domande o le frasi inserite nella barra di ricerca, e restituisce i risultati più pertinenti e rilevanti da un vasto indice di pagine web. È composto da:

- **FORMATTATORE DI RISULTATI**: parte del sistema di Google che si occupa di **comporre**, cioè inserisce nella pagina i risultati ottenuti dalla ricerca, come:: inserzioni pubblicitarie (se qualcuno ha pagato), news, mappe, immagini e risultati organici valutati da PageRank.
- **ENGINE - PAGERANK**: algoritmo di Google che **classifica** le pagine web, valutando l'importanza della pagina in base a: popolarità della pagina (aumenta quando un link compare tante volte in altri link), posizione (in alto a sinistra) e dimensione dei termini cercati. Una pagina con Rank maggiore è giudicata più importante nel contesto della ricerca effettuata, non è un fattore globale!
- **INTERFACCIA UTENTE - SEARCH BOX**: consente di **vedere immediatamente** alcuni suggerimenti di ricerca basati su ciò che si ha

11

scritto e senza apparentemente inviare informazioni al server; succede grazie alla tecnologia *AJAX* che consente di inviare e ricevere dati al/dal server di nascosto e poi di poterli mostrare all'utente.



1.4 IL WEB 2.0

Esistono svariati significati del termine “**Web 2.0**”:

1. Tim O'Reilly, nel 2004, si riferisce all'evoluzione del web e all'introduzione di servizi di seconda generazione
2. interconnessione con persone e non con macchine
3. utilizzo di tecnologie client-side evolute: **Ajax** (Asynchronous JavaScript and XML), insieme di tecniche e metodologie di sviluppo software per la realizzazione di applicazioni web interattive, basandosi su uno scambio di dati in background fra web browser e server
4. utilizzo dei Web Services: **API** (Application Programming Interface)
5. il web ha sempre avuto tutte queste caratteristiche

12

Le **tecnologie server side** rappresentano

l'insieme di strumenti e linguaggi che risiedono ed eseguono direttamente sul server, con

l'obiettivo di fornire servizi e contenuti agli utenti.



Queste tecnologie sfruttano le risorse del server stesso, come la CPU, la memoria RAM e lo spazio su disco, per elaborare richieste e generare risposte dinamiche.

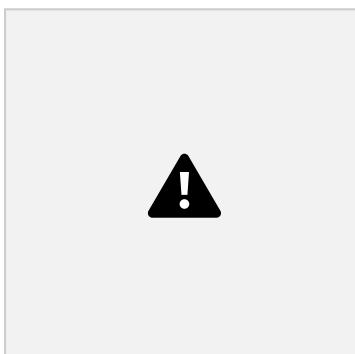
Esempi di tecnologie server side includono server web come *Apache*, *NGINX* e *IIS*, database come *MySQL* e *Cassandra*, e linguaggi di scripting come *PHP*, *Perl*, *Ruby* e *Python*. Questi strumenti lavorano insieme per gestire il flusso di dati, eseguire calcoli complessi e garantire che le applicazioni web funzionino correttamente.

Le **applicazioni web** sono implementate tramite server web che utilizzano tecnologie server side. Queste applicazioni si basano su database per la gestione dei dati e su linguaggi di scripting per la logica applicativa. L'integrazione di questi elementi consente la creazione di piattaforme interattive e dinamiche accessibili via internet.

I **servizi di hosting** offrono ambienti completi che includono tutte le tecnologie server side necessarie per lo sviluppo e la gestione di applicazioni web. Aziende di hosting mettono a disposizione server configurati con strumenti come *NGINX*, *PHP*, *MySQL*, *Ruby* e *PostgreSQL*, permettendo agli sviluppatori di concentrarsi sulla creazione di contenuti e funzionalità senza preoccuparsi della gestione dell'infrastruttura sottostante.

13

1.1 COS'È HTTP?



HTTP (Hyper Text Transfer Protocol) è un protocollo di richiesta e risposta (request-response), che consente la trasmissione di informazioni (pagine web, file multimediali, ecc.) sul web. Quando si visita un sito web, il browser invia una richiesta HTTP al server che ospita quel sito, e il server

risponde inviando le risorse richieste, come il codice HTML, le immagini e i file CSS.

HTTP è un protocollo a livello applicativo (per quanto riguarda il modello ISO/OSI) e utilizza una tipica architettura client-server. Un server HTTP generalmente resta in ascolto delle richieste del client sulla porta 80 usando il protocollo TCP a livello di trasporto.



14

Possiamo dire che il **Web** (WWW) è nato dall'insieme di diverse tecnologie, tra cui le principali sono HTTP, URL e HTML.



1.2 STORIA DELL'HTTP

La prima versione dell'HTTP, la 0.9, risale alla fine degli anni 1980 sviluppata da Tim Berners-Lee al CERN di Ginevra per la condivisione delle informazioni tra la comunità dei fisici delle alte energie. La prima versione effettivamente disponibile del protocollo fu la HTTP/1.0, ma quando il WWW conobbe un successo crescente divennero evidenti alcuni limiti di tale versione del protocollo, in particolare:

- l'impossibilità di ospitare più siti web sullo stesso server
- il mancato riuso delle connessioni disponibili;
- l'insufficienza dei meccanismi di sicurezza.

Il protocollo venne quindi esteso nella versione HTTP/1.1.

È ben noto che l' HTTP sia un **protocollo di tipo request-response senza stato**

(*stateless*). La semplicità del protocollo HTTP lo rende molto scalabile ma inefficiente e non adatto per le applicazioni web altamente interattive o in tempo reale, oggi sempre più diffuse.

D'altronde l'HTTP è stato progettato per la condivisione di documenti e non per la creazione di questo tipo di applicazioni.

Prima della versione 1.1 del protocollo, ogni richiesta al server comportava una nuova connessione, mentre dalla versione 1.1 questo aspetto è stato migliorato con l'introduzione delle **connessioni persistenti** (o permanenti) che permettono al browser di riutilizzare la stessa connessione per il caricamento di più immagini, script, ecc...

15

1.3 FUNZIONALITÀ DEL HTTP 2.0

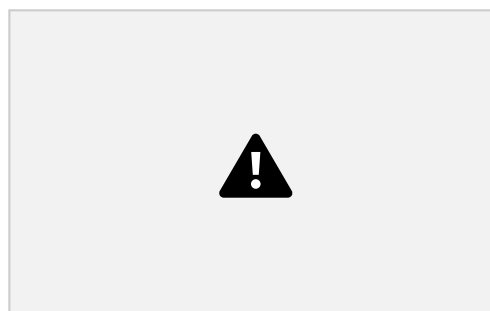
Esistono due tipi di connessione persistente:

- connessione persistente **incanalata** (default);
- connessione persistente **non incanalata**.

Nella versione non incanalata il client passa una nuova richiesta al server solo quando la precedente è stata chiusa. Mentre la versione incanalata inserisce tutte le richieste del client in una coda chiamata pipeline, la

quale invia le richieste in ordine di arrivo (FIFO) al server man mano che chiude le precedenti.

Quest'ultima modalità permette di ridurre i tempi di attesa tra l'invio delle richieste, migliorando l'efficienza del trasferimento dei dati quando sono necessarie più risorse. La nuova versione del protocollo, HTTP/2, è stata pubblicata nel maggio 2015.



Funzionalità del 2.2 che incrementano le performance:

- **Multiplexing**: nella stessa connessione è possibile avere richieste e risposte multiple che vengono "ricostruite" nell'host di destinazione. Questo permette di fare operazioni in parallelo e quindi di ridurre la latenza = possibilità di poter fare più richieste in una volta sola, ciò porta ad un grosso risparmio di tempo (nel 1.1 si poteva ottenere una sola risorsa alla volta e *per esempio per caricare un sito si dovevano fare numero*

richieste...una per il sito, una per ogni immagine, una per lo stile e così via);

- **Compressione headers:** anche gli header HTTP, e non solo il body, sono compressi e il consumo di banda è notevolmente ridotto;
- Supporto per **operazioni di push** da parte del server, che può inviare dati al client prima che questo ne faccia richiesta (esempio: se una pagina era 404 ovvero not found e ora diventa disponibile mi contatta il server per avvisarmi della disponibilità, non c'è bisogno che rieffettuo la richiesta).

16

1.4 TIPI DI TRASMISSIONE HTTP

L'HTTP, inoltre, è stato progettato per essere **half-duplex** il che significa che la trasmissione dei dati è consentita in una sola direzione alla volta. Per superare queste lacune dell'HTTP gli sviluppatori hanno creato alcune soluzioni o hack.

Tra questi troviamo:

- il polling;
- il long polling;
- lo streaming (o WebSocket).

Polling (metodo sincrono): Il client invia una richiesta al server, e il server risponde immediatamente con i dati disponibili. Se non ci sono dati nuovi, la risposta conterrà informazioni vuote o lo stato attuale. Il client quindi attende un periodo di tempo (ad esempio, 5 secondi) e ripete la richiesta. Ha dalla sua parte la semplicità di utilizzo e di implementazione, è però poco efficiente in termini di utilizzo di risorse di rete.

Long Polling (metodo asincrono): Il client invia una richiesta al server, ma il server non risponde immediatamente. Mantiene la connessione aperta fino



a quando non ha dati da restituire. Quando il server invia una risposta, il client invia una nuova richiesta, e il ciclo continua. È più efficiente rispetto al polling tradizionale poiché riduce il numero di richieste inutili, è però meno intensivo e richiede una gestione accurata delle connessioni aperte.

17

Per soddisfare la crescente esigenza delle applicazioni web di effettuare comunicazioni bidirezionali in modo standard, è stato introdotto il concetto di **WebSocket**. Anche se ogni linguaggio di programmazione ha la sua implementazione, noi ci concentreremo sull'implementazione in Java (tramite l'API JSR 356) e su un client in JavaScript.

I WebSocket sono un protocollo che permette una comunicazione asincrona e bidirezionale (**full-duplex**) su una connessione TCP. Sebbene la connessione parta con una richiesta HTTP, **non si tratta di una connessione HTTP**. WebSocket è stato originariamente proposto nella specifica HTML5, ma poi è stato spostato in un documento separato (RFC 6455 e la WebSocket API in JavaScript) per concentrarsi unicamente su questo protocollo.

Ogni connessione WebSocket inizia con una richiesta HTTP. Tuttavia, questa richiesta include un'intestazione speciale chiamata Upgrade, che indica che il client desidera passare a un protocollo diverso, in questo caso WebSocket. Una volta completato l'aggiornamento, la connessione WebSocket viene stabilita tra client e server sulla stessa connessione iniziale, consentendo la comunicazione bidirezionale.

I browser moderni supportano WebSocket e forniscono un'API in JavaScript che permette di connettersi agli endpoint, inviare messaggi e gestire eventi come l'apertura della connessione, la ricezione di messaggi e la chiusura della connessione.

Alcuni esempi di utilizzo di WebSocket includono:

- Applicazioni di chat
- Giochi multiplayer

- Trading di azioni o applicazioni finanziarie
- Collaborazione su documenti in tempo reale
- Applicazioni di social networking

1.5 GLI IDENTIFICATORI

Il protocollo HTTP fa uso di **identificatori**, gli **URI** (Uniform Resource Identifier), creati per accedere a risorse fisiche o astratte sulla rete Internet, che possono essere di vario tipo: siti web, mittenti o destinatari di e-mail, immagini, file audio e così via. Le applicazioni utilizzano un identificatore univoco per interagire con una risorsa o per recuperarne i dati.

Essi si dividono in:

- **URL** (*Uniform Resource Locator*): identificano una risorsa immediatamente utilizzabile da un programma. Sono composti da: protocol, host name, porta, nome del percorso

Caratteristiche:

- Contengono tutte le informazioni necessarie per accedere ad una risorsa; - Fragili, in quanto possono perdere il loro “valore” se soggetti a modifiche;

- **URN** (*Uniform Resource Name*): nome stabile e definitivo di una risorsa, che fornisce un'informazione certa ed affidabile

Caratteristiche:

- Devono essere trasformati in URL da un apposito servizio

Tra questi tre termini può sorgere confusione. L'URL è usato per indicare dove si trova una risorsa, e lo si usa anche quando si naviga in Internet per raggiungere determinate pagine web. Al contrario, l'**URI** è indipendente dal luogo e designa una risorsa in maniera univoca. Se gli URL sono noti principalmente sotto forma di indirizzi web, un URN può anche presentarsi come ISBN, per identificare ad esempio un libro.

URL e URN seguono la sintassi degli URI; sono quindi dei sottoinsiemi degli URI.



19

1.6 MESSAGGIO DI RICHIESTA: HTTP REQUEST

Come detto prima, il protocollo HTTP funziona su un'architettura client-server in cui due entità, il Client e il Server, si scambiano messaggi. L'HTTP Request è un messaggio testuale inviato dal client al server HTTP ed è formato da 3 elementi:

- riga di richiesta;
- intestazione HTTP (header);
- corpo del messaggio (body).

La request segue la seguente sintassi:



La **riga di richiesta** contiene:

- Il **metodo** (sempre scritto in maiuscolo):
 - CONNECT, DELETE, GET, HEAD, POST, PUT, OPTIONS, TRACE;
- L'**URI** (o **URL**): identificativo della risorsa richiesta dal server;
- La **versione** (version) che può assumere i valori HTTP/1.0 o HTTP/1.1 .

Un esempio di riga di richiesta HTTP è la seguente:



L'intestazione HTTP (**header**) contiene tutte le informazioni necessarie per l'identificazione del messaggio. È formata da diverse righe, ciascuna delle quali rispetta il formato:

20

Un esempio di header HTTP è il seguente:



Il corpo del messaggio (**body**) contiene le informazioni che si vogliono trasferire al server.

Un esempio di HTTP request completa è il seguente:



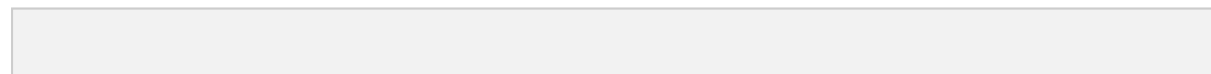
1.7 MESSAGGIO DI RISPOSTA: HTTP RESPONSE

Una volta che il server HTTP ha ricevuto una richiesta dal client, inizia la fase di risposta per fornire la risorsa richiesta. Questa fase può includere, ad esempio, l'esecuzione di uno script in tecnologie server-side. Fatto ciò, il server risponde al client con una HTTP response.

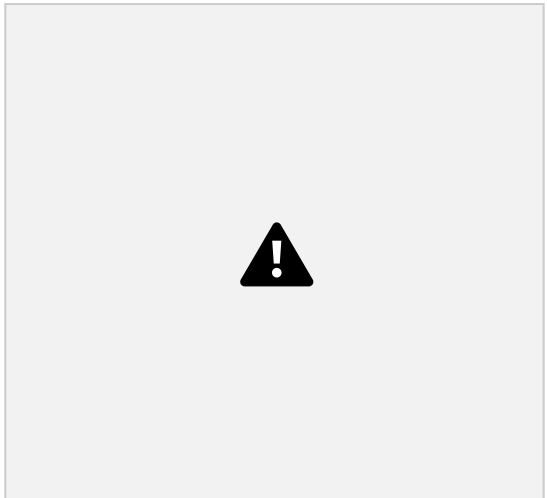
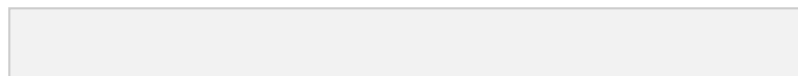
La **response** HTTP mantiene la stessa struttura della request, con la sola differenza che le risposte iniziano con una riga di stato anziché una riga di richiesta.

La response è quindi formata da:

- una **riga iniziale** con versione del protocollo HTTP e stato; •
- un'intestazione (**header**);
- un corpo (**body**);



Un esempio di **riga iniziale di stato** è la seguente:

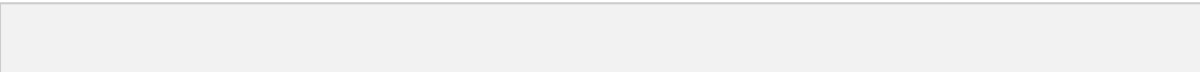


Gli **header** della response sono formati, come nella request, da insiemi di coppie formate da un nome separato dal valore dai due punti (:), che specificano le caratteristiche del messaggio.

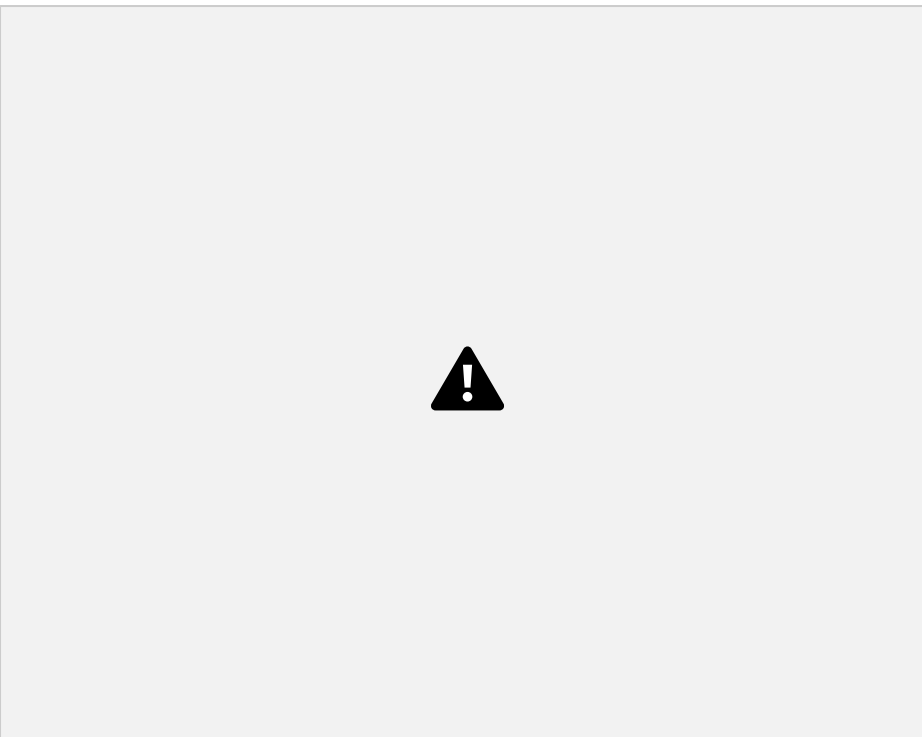
Qui un breve riassunto con alcuni tipi di header:

- **Header Generali:** Informazioni sulla comunicazione generale (es. *Date*, *Connection*);
- **Header di Richiesta:** Informazioni specifiche del client (es. *User-Agent*, *Authorization*);
- **Header di Risposta:** Informazioni inviate dal server (es. *Content-Type*, *Set-Cookie*);
- **Header di Entità:** Metadati relativi al contenuto (es. *Content-Encoding*, *Last-Modified*);
- **Header di Sicurezza:** Migliorano la sicurezza della comunicazione (es. *Strict-Transport-Security*, *X-Frame-Options*).

La sezione **body** della response contiene le informazioni richieste dal client.



Un esempio di HTTP response completa è il seguente:



I **codici di stato** sono numeri utilizzati nei protocolli di comunicazione, come l'HTTP, per indicare l'esito di una richiesta tra un client e un server. Ogni codice è composto da tre cifre e fornisce informazioni chiave sul risultato dell'operazione, permettendo al client di capire se la richiesta è stata completata con successo o se ci sono stati problemi. I codici di stato sono suddivisi in categorie a seconda della prima cifra, che indicano il tipo di risposta: successi, reindirizzamenti, errori client o errori server. Comprendere questi codici è fondamentale per diagnosticare problemi nelle comunicazioni di rete e ottimizzare l'interazione tra client e server.

Di seguito una tabella riassuntiva con alcuni dei principali codici di stato.



1.8 LIMITI DEL HTTP

Il protocollo HTTP presenta alcuni limiti:

- **Stateless:** Ogni richiesta HTTP è indipendente dalle precedenti, ossia ciascuna è considerata un'operazione atomica, singola e autonoma. Ad esempio, se visitiamo due

pagine, ciascuna richiesta è trattata separatamente. Non esiste alcun meccanismo intrinseco che permetta al protocollo HTTP di collegare una richiesta con quella precedente.

- **Tecnologia pull:** HTTP è basato su un modello pull, dove il client deve sempre richiedere esplicitamente le informazioni. Non è possibile che il server inizi una comunicazione con il client per inviare dati non richiesti.
- **Assenza di contesto della richiesta:** HTTP non fornisce informazioni sull'origine della richiesta. Non c'è un modo standard per determinare se una richiesta è stata generata da un click su un link o da una visita diretta a una pagina.

1.9 PROTOCOLLO IP

Il protocollo IP appartiene alla famiglia dei protocolli di rete e ha il compito di identificare i nodi della rete assegnando a ciascuno un indirizzo IP. È responsabile dell'indirizzamento e della frammentazione dei pacchetti nelle reti digitali. Insieme al protocollo TCP (Transmission Control Protocol), l'indirizzo IP costituisce la base di Internet.

Per trasmettere un pacchetto dal mittente al destinatario, l'IP definisce una struttura che include informazioni sull'origine e la destinazione dei dati. Queste informazioni sono contenute nell'header IP, separate dal contenuto effettivo del pacchetto. Ogni indirizzo IP è composto da 32 bit suddivisi in 4 gruppi da 1 byte, con valori che vanno da 0 a 255, permettendo di generare indirizzi compresi tra 0.0.0.0 e 255.255.255.255.

Dietro ogni nome di dominio (esempio, www.unife.it) si nasconde un indirizzo IP. Cose importanti da sapere sugli indirizzi IP:

25

- **DNS (Domain Name System):** È il sistema che converte i nomi di dominio in indirizzi IP e viceversa. Funziona tramite un database distribuito di server DNS e ha una struttura gerarchica ad albero rovesciato, divisa in domini.
- **Indirizzi IP riservati:** Alcuni indirizzi IP sono riservati e non possono essere assegnati a dispositivi di rete. I più noti sono:
 - **127.0.0.1:** Identifica sempre la macchina locale (localhost).
 - **0.0.0.0:**

Rappresenta una connessione a Internet non specificata.

Inoltre, se il client riceve una risposta protetta da password, viene restituito il codice 401 per indicare che non è autorizzato a visualizzare la risorsa richiesta.

1.10 W3C

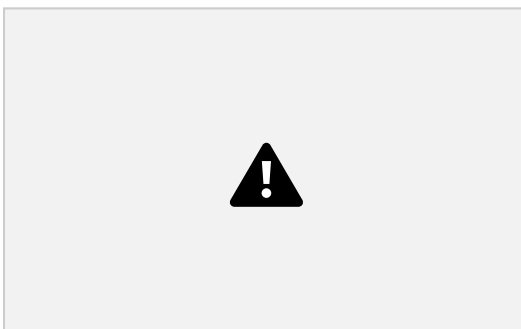
Il **World Wide Web Consortium** (W3C) è stato fondato nel 1994 da Tim Berners-Lee con l'obiettivo di "condurre il World Wide Web verso il suo pieno potenziale, sviluppando protocolli e linee guida che garantiscano una crescita sostenibile nel lungo termine" (dallo statuto). In sostanza, il W3C è un consorzio internazionale in cui organizzazioni, membri, personale permanente e pubblico collaborano per sviluppare gli standard del web.

Ogni standard del W3C passa attraverso diverse fasi evolutive prima di diventare definitivo:

- **Working Draft (WD)**: Documento rilasciato dal W3C per la revisione da parte della comunità del web; si tratta di una bozza iniziale.
- **Candidate Recommendation (CR)**: Documento che ha subito ampie revisioni e soddisfa i requisiti tecnici del gruppo di lavoro.
- **Proposed Recommendation (PR)**: Documento maturo, sottoposto a revisioni approfondite e con verificate possibilità di implementazione, in attesa dell'approvazione finale.
- **W3C Recommendation (REC)**: Documento che rappresenta uno standard ufficiale per il W3C, raccomandando l'uso e la diffusione massima.

26

2.1 COS'È IL MODELLO CLIENT - SERVER?



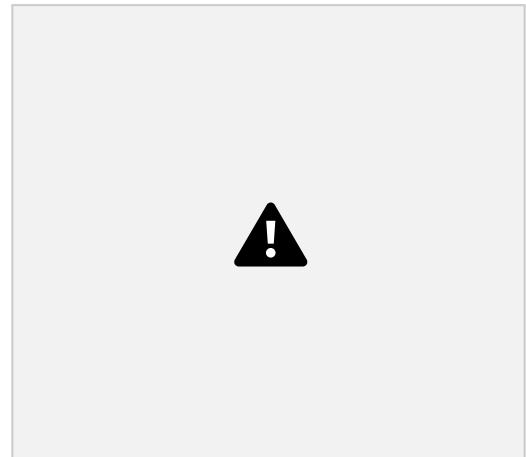
Alle fondamenta della comunicazione in rete c'è l'**architettura client-server**, un modello che organizza la distribuzione e l'accesso ai servizi tra due entità principali: il **client**, che richiede un servizio o una risorsa, e il **server**, che lo fornisce. Questo sistema, ampiamente adottato per il funzionamento di siti web, applicazioni

mobili e servizi di rete, permette una gestione centralizzata delle risorse e facilita lo scambio di dati tra dispositivi connessi. L'architettura client-server è il motore silenzioso dietro molte delle interazioni digitali quotidiane, dalla navigazione web all'uso di servizi cloud, rendendo possibili comunicazioni rapide ed efficienti.

2.2 COME FUNZIONA?

Questo modello di comunicazione è caratterizzato da un'interazione **diretta** e **asimmetrica**:

- Il **client** avvia esplicitamente la comunicazione con il server, specificando in modo chiaro e non ambiguo il servizio richiesto.
- Il **server** è progettato per gestire più client contemporaneamente, rimanendo costantemente in attesa di nuove richieste per poterle elaborare.



Il client:

- Consente all'utente di inviare una richiesta di informazioni al server. 27

• Formatta la richiesta in modo comprensibile per il server. • Elabora la risposta del server per renderla leggibile all'utente. Il server:

- Riceve e processa la richiesta proveniente dal client.
- Risponde inviando al client le informazioni richieste.

2.3 PROTOCOLLI DI COMUNICAZIONE: TIPI DI ITERAZIONE

Esistono due principali tipi di comunicazione in rete:

- **Connection-oriented**: viene stabilito un canale di comunicazione virtuale prima di avviare lo scambio di dati (comunicazioni telefoniche).
- **Connectionless**: non viene creata una connessione virtuale, e i dati vengono trasmessi

tramite uno scambio di messaggi diretto (sistema postale).

La scelta tra i due modelli dipende dal tipo di applicazione e dalle caratteristiche del livello di comunicazione sottostante.

- **TCP** è orientato alla connessione, garantisce l'affidabilità (reliable) e preserva l'ordine di invio dei messaggi.
- **UDP** è senza connessione, non garantisce l'affidabilità (non reliable) e non preserva l'ordine dei messaggi.

2.4 MODELLI PULL E PUSH

I **modelli di comunicazione pull e push** sono due approcci utilizzati per la trasmissione di dati tra client e server. La differenza principale risiede nel modo in cui i dati vengono richiesti e inviati: nel modello pull è il client a richiedere le informazioni, mentre nel modello push è il server che invia i dati al client quando disponibili. Entrambi i modelli hanno vantaggi e svantaggi a seconda delle applicazioni e delle esigenze specifiche. Vediamo le specifiche di questi due modelli.

Nel modello **Pull**, il client richiede esplicitamente un servizio al server. Ciò può avvenire in due modi:

- **Sincrono**: il client si blocca e attende la risposta del server.
- **Asincrono**: il client non si blocca, ma verifica periodicamente (polling) se la risposta è arrivata.



In questo modello, il server rimane passivo fino a quando un client non richiede un contenuto informativo.

VANTAGGI:

- **Full automation:** è possibile automatizzare completamente il processo, configurando una connessione verso il server senza intervento dell'utente.
- **Maggiore scalabilità:** la struttura centralizzata offre la possibilità di gestire molti client, anche se il sistema potrebbe subire colli di bottiglia al livello del server.

SVANTAGGI:

- **Vulnerabilità agli attacchi DoS:** essendo il server centrale, può essere soggetto ad attacchi Denial of Service (DoS) che lo inondano di richieste, mettendolo offline.
- **Gestione complessa dei contenuti:** il server deve memorizzare i dati per rispondere a ogni richiesta e verificare se il client è autorizzato a riceverli.

29

Nel modello **Push**, invece, il client segnala al server il proprio interesse a ricevere un servizio, e il server invia le informazioni al client appena disponibili, senza che il client debba fare ulteriori richieste.



VANTAGGI:

- **Maggiore controllo:** le azioni sono sotto il controllo dell'utente, che può monitorare l'invio delle informazioni.
- **Semplicità:** l'architettura è più semplice, poiché si basa su un singolo utente che invia comandi verso macchine remote, rendendo gli strumenti push mediamente più facili da implementare rispetto al modello pull.
- **Meno richieste:** rispetto al modello pull, il push riduce la necessità di continue richieste, proteggendo il sistema da attacchi DoS.

SVANTAGGI:

- **Mancanza di automazione completa:** è necessario l'intervento manuale per configurare nuovi host.

- **Minore scalabilità:** il modello push presenta difficoltà nel gestire un numero crescente di host, limitando la sua capacità di scalare in modo efficiente.

2.5 STATO

L'interazione tra un Client e un Server può essere classificata in due modalità principali:

- **Stateful:** lo stato dell'interazione viene mantenuto, il che significa che ogni messaggio inviato è legato ai precedenti. Il server conserva informazioni sul contesto della comunicazione.
- **Stateless:** non viene mantenuta alcuna traccia dello stato dell'interazione, quindi ogni messaggio è completamente indipendente dagli altri. Ogni richiesta del client viene trattata come una nuova e isolata dal contesto delle precedenti.

30

Generalmente, il compito di memorizzare lo stato dell'interazione è a carico del server.

- Un **server stateful** offre maggiore efficienza, con messaggi di dimensioni più ridotte e tempi di risposta più rapidi, grazie alla continuità nel flusso di comunicazione.
- Un **server stateless**, d'altra parte, è più affidabile in presenza di malfunzionamenti, in particolare problemi di rete, ed è più semplice da progettare e mantenere.

Un esempio comune di protocollo stateless è l'**HTTP**, che tratta ogni richiesta come indipendente dalle altre.

2.6 MODELLO ITERATIVO E CONCORRENTE

Sul lato client, l'esecuzione concorrente è gestita dal multitasking del sistema operativo. Sul server, invece, la concorrenza è cruciale per servire più client contemporaneamente. Esistono due principali modelli di server: il **modello iterativo** e il **modello concorrente**.

Nel modello **iterativo** (o sequenziale), il server elabora una richiesta alla volta.

Caratteristiche:

- Basso consumo di risorse.
- Semplicità progettuale.

Funzionamento:

- Se il server è libero, il client viene servito immediatamente.
- Se il server è occupato:
 - Se c'è spazio in coda, il client attende.
 - Se la coda è piena, la richiesta viene rifiutata.

31

Problemi:

- Gestione complessa della coda.
- Lunga attesa per i client.
- Vulnerabilità agli attacchi DoS.

Prestazioni:

Il tempo di risposta complessivo (T_r) è dato dalla formula:

$$T_r = T_s + 2T_c + T_q$$

- **T_r** : ritardo totale tra l'invio della richiesta e la risposta.
- **T_s** : tempo di elaborazione della richiesta.
- **T_c** : tempo medio di comunicazione.
- **T_q** : tempo medio di attesa in coda.

Con lunghe code di richieste, il tempo di risposta può aumentare notevolmente rispetto al tempo di elaborazione effettivo.

Nel modello **concorrente**, invece, il server gestisce più richieste simultaneamente creando un nuovo processo o thread per ciascuna richiesta.

Caratteristiche:

- Viene creato un processo/thread separato per ogni richiesta client.
- Alto consumo di risorse.

- Maggiore complessità progettuale.

Funzionamento:

La concorrenza è ottenuta tramite un parallelismo apparente (**time sharing**) nella gestione delle connessioni, ma non nella trasmissione dei dati.

Problemi:

Gestione complessa, poiché il server deve mantenere attivi tutti i canali di comunicazione con i vari client.

32

Prestazioni:

La concorrenza migliora il tempo di risposta complessivo (T_r), calcolato come:

$$T_r = T_s + 2T_c + T_q + T_g$$

- **T_c** : tempo medio di comunicazione.
- **T_q** : tempo medio di attesa in coda (spesso trascurabile).
- **T_g** : tempo di generazione del processo o thread di servizio.

Rispetto al modello iterativo, il modello concorrente consente **tempi di risposta più rapidi**, soprattutto in presenza di numerose richieste simultanee. Tuttavia, introduce una **maggiore complessità** nella gestione del sistema e un **consumo più elevato** di risorse.

2.7 TECNOLOGIE SERVER-SIDE

Per realizzare un'applicazione web sono necessari i seguenti componenti:

- Web Server

- Database Server
- Linguaggio di programmazione

Ma cos'è un **Web Server**? Un web server è un software, in esecuzione su un server, progettato per gestire le richieste di contenuti web da parte dei client, solitamente i browser. La sua funzione principale è quella di ricevere, elaborare e rispondere a queste richieste inviando i dati richiesti, come pagine HTML, immagini o file multimediali.

Una delle soluzioni più comuni è l'uso dello stack **LAMP**, un acronimo che rappresenta i suoi componenti principali:

- GNU/Linux: il sistema operativo.
- Apache: il server web.
- MySQL: il sistema di gestione di database (DBMS).
- PHP, Perl, o Python: i linguaggi di programmazione.

33

Questi componenti lavorano insieme in modo interdipendente, motivo per cui questa combinazione di software è nota come **LAMP Stack**.

Nel dettaglio, il sistema operativo Linux fornisce la base per l'esecuzione del server web Apache. Tuttavia, Apache da solo non può elaborare contenuti dinamici; per questo si affida a un interprete, come PHP, che è in grado di accedere al database MySQL. Quando Apache riceve una richiesta che richiede contenuti dinamici, inoltra il codice all'interprete PHP, che elabora le informazioni del database e restituisce i risultati ad Apache, il quale poi li mostra nel browser dell'utente.

La comunicazione tra server e client avviene tramite il protocollo HTTP, che utilizza la porta 80. La versione sicura di HTTP, **HTTPS**, usa la porta 443 ed è basata su SSL (Secure Sockets Layer) per garantire la trasmissione sicura dei dati.

CHE DIFFERENZA C'È TRA HTTP E HTTPS?

La differenza principale tra HTTP e HTTPS risiede nella sicurezza della trasmissione dei dati.

1. HTTP (Hypertext Transfer Protocol):

- È un protocollo per la trasmissione di informazioni su una rete, principalmente utilizzato per la comunicazione tra un browser web (client) e un server.
- I dati inviati tramite HTTP non sono criptati, il che significa che possono essere

intercettati e letti da terzi durante il transito. Questo rende HTTP meno sicuro per la trasmissione di informazioni sensibili, come password o dati personali.

- HTTP utilizza la porta **80** per la comunicazione.

2. **HTTPS** (Hypertext Transfer Protocol Secure):

- HTTPS è la versione sicura di HTTP. Utilizza **SSL** (Secure Sockets Layer) o **TLS** (Transport Layer Security) per criptare i dati trasmessi tra il client e il server.

34

- La crittografia assicura che i dati siano illeggibili a chiunque cerchi di intercettarli durante il transito, proteggendo così le informazioni sensibili come dati di pagamento e credenziali di login.
- HTTPS utilizza la porta **443** per la comunicazione sicura.
- Viene anche impiegato un certificato digitale per autenticare il sito web, garantendo agli utenti che stanno comunicando con il server legittimo e non con un impostore.

In sintesi, **HTTP** non offre protezione contro intercettazioni o attacchi, mentre **HTTPS** garantisce la sicurezza dei dati trasmessi attraverso la crittografia e l'autenticazione del sito.

2.8 PAGINE STATICHE E PAGINE DINAMICHE

Le pagine web si dividono in due categorie principali: **statiche** e **dinamiche**.

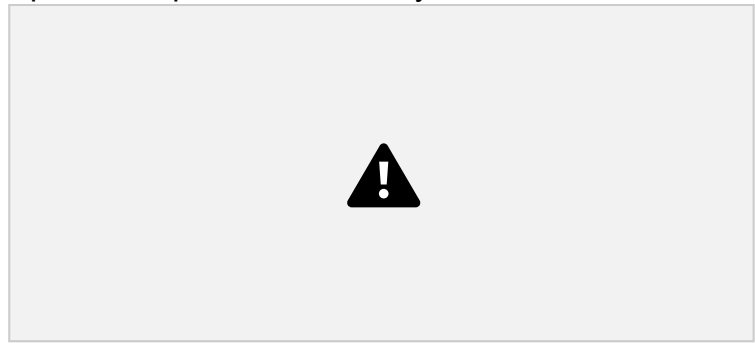
Le pagine **statiche** sono composte esclusivamente da codice **HTML** e definiscono con precisione il testo, la grafica e le immagini da visualizzare. Il contenuto di queste pagine è fisso e non può essere modificato dinamicamente. Inoltre, il server restituisce sempre lo stesso contenuto in risposta a tutte le richieste **HTTP**.

Come viene recuperata una pagina statica:

- L'URL inserito dall'utente viene interpretato dal browser (user agent) e inserito nella richiesta HTTP.
- Il web server sull'host accoda il percorso della richiesta a quello della sua directory principale (root directory).

- Il server risponde inviando il file corrispondente presente nel filesystem.

Le pagine **dinamiche**, invece, contengono codice HTML insieme a istruzioni scritte in linguaggi di scripting server-side (come **PHP** o **Python**). Il server web interpreta queste istruzioni e genera il



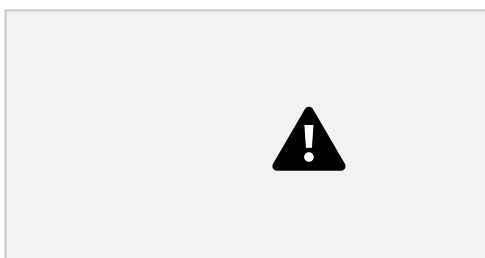
35

contenuto della pagina in tempo reale, in base alle condizioni o parametri specifici. Di conseguenza, il contenuto della risposta HTTP può variare per ogni richiesta, rendendo le pagine dinamiche molto più flessibili. Vantaggi delle pagine dinamiche:

- Possono ampliare le funzionalità del sito web.
- Sono ideali per siti che richiedono aggiornamenti frequenti e rapidi.
- Permettono di aggiornare, aggiungere o eliminare contenuti in tempo reale.
- Consentono di integrare motori di ricerca interni.
- Possono proteggere sezioni del sito con username e password. Come

viene recuperata una pagina dinamica:

Prima di inviare la risposta al client, il server esegue lo script (ad esempio, PHP o Python) per generare la pagina. Il risultato viene poi inviato al browser sotto forma di HTML.



2.9 SUBDIRECTORY

Comunemente, un server può ospitare più siti web e ci sono due principali modalità per farlo:

1. **Subdirectories:** Questa soluzione è semplice e veloce, particolarmente adatta per lo sviluppo locale. Tuttavia, in un ambiente di produzione, può causare problemi legati alla condivisione del contesto, come la gestione dei cookie, poiché tutti i siti condividono lo stesso host.

- Per implementare questa soluzione, si crea una directory separata per ogni sito all'interno della directory principale del server (es. /var/www/).

2. **Virtual Hosts:** Consente di ospitare più siti web su uno stesso server, utilizzando nomi di dominio distinti, e talvolta anche lo stesso indirizzo IP. Questa soluzione è ideale per gestire più siti separatamente, evitando i problemi di condivisione che si verificano con le subdirectory.

36

Nel server **Apache**, i siti configurabili tramite virtual hosts si trovano nella directory **sites-available/**. Ogni file in questa directory rappresenta un sito, e all'installazione, è presente un file predefinito chiamato **000-default.conf**.

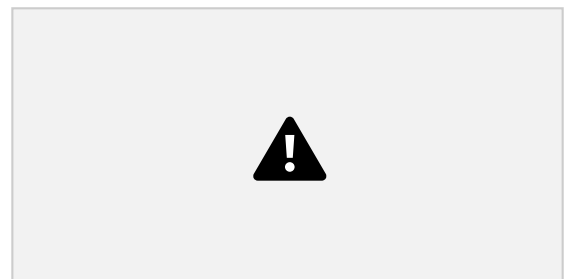
2.10 SERVIZI DI HOSTING

SHARED HOSTING

È la soluzione di hosting più comune ed economica, in cui le risorse del server sono condivise tra più utenti.

Caratteristiche:

- Risorse limitate (CPU, memoria, spazio disco, banda).
- Configurazioni e accessi limitati.
- Nessuna manutenzione richiesta da parte dell'utente.
- Vantaggio principale: basso costo.



VIRTUAL PRIVATE SERVER (VPS)

Una soluzione più avanzata e professionale rispetto allo shared hosting, con risorse virtualmente separate per ogni utente.

Caratteristiche:

- Risorse condivise ma con maggiore controllo.
- Nessuna limitazione sulla configurazione del server.
- Richiede manutenzione da parte dell'utente.
- Prezzo moderato.

DEDICATED SERVER

Il Dedicated Server è destinato a chi ha esigenze elevate e richiede risorse totalmente dedicate.

Caratteristiche:

- Risorse completamente dedicate (CPU, memoria, spazio). 37
- Nessuna limitazione di configurazione.
- Generalmente molto costoso.
- Richiede manutenzione completa a carico dell'utente.

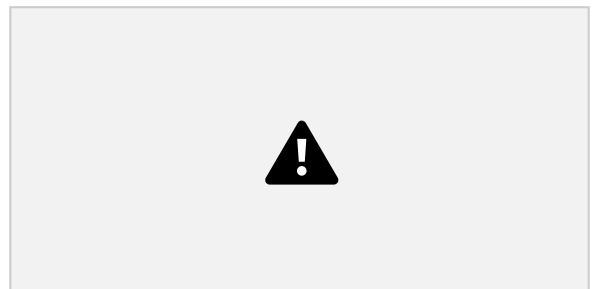
CLOUD HOSTING

Il Cloud Hosting offre una soluzione flessibile che

media tra costi e scalabilità. Caratteristiche:

- Risorse limitate ma facilmente espandibili.
- Nessuna limitazione nella configurazione.
- Richiede manutenzione da parte dell'utente.
- Necessita competenze tecniche specifiche per la configurazione. Vantaggio principale:

l'utente non deve preoccuparsi della gestione fisica del server, che è a carico del provider del servizio cloud.



Componenti del Cloud Hosting

Il **Cloud Hosting** ha rivoluzionato il modo in cui le organizzazioni gestiscono le loro infrastrutture IT, offrendo un approccio flessibile e scalabile per soddisfare le esigenze moderne. Alla base di questa innovativa architettura si trovano diversi componenti fondamentali, ciascuno con un ruolo specifico nel garantire un funzionamento efficiente e performante.

- **Storage:** si occupa della memorizzazione dei dati.
- **Calcolo:** gestisce la virtualizzazione dei processi.
- **Controller:** coordina l'intera infrastruttura cloud.
- **IaaS** (Infrastructure As A Service): offre flessibilità senza l'onere della gestione

dell'hardware.

- **PaaS** (Platform As A Service): consente all'utente di concentrarsi esclusivamente sullo sviluppo del software e il suo caricamento sulla piattaforma.
- **SaaS** (Software As A Service): permette di utilizzare il software direttamente sulla piattaforma, senza preoccuparsi di installazioni o aggiornamenti.



38

3.1 VCS, DEFINIZIONE E CARATTERISTICHE

Git è un sistema di controllo di versione (VSC) distribuito, moderno, open source e gratuito, progettato per gestire in modo efficiente e veloce i dati di progetto. In un sistema distribuito come Git, ogni sviluppatore possiede una copia locale completa del repository, il che permette di salvare le modifiche a livello locale in qualsiasi momento. Anche se esiste un server remoto che ospita il repository condiviso tra tutti gli sviluppatori, la temporanea indisponibilità del server non impedisce di continuare a lavorare. Le modifiche possono essere registrate nel repository centrale non appena il server torna disponibile.

Per **VCS** si intende un sistema di controllo di versione, ovvero una combinazione di tecnologie e procedure utilizzate per tenere traccia nel tempo delle modifiche fatte ad un file o ad un insieme di file per recuperare una vecchia versione di un determinato file.

Lo scopo principale di un sistema di un VCS è garantire una gestione efficiente e sicura delle modifiche a un progetto, fornendo le seguenti caratteristiche fondamentali:

- **Reversibilità**: permette di tornare a qualsiasi punto della storia del codice, facilitando il recupero di versioni precedenti.
- **Concorrenza**: consente a più sviluppatori di lavorare contemporaneamente sullo stesso progetto, integrando facilmente le modifiche apportate da diversi membri del team.
- **Annotazione**: offre la possibilità di allegare note e spiegazioni alle modifiche

effettuate, chiarendo le motivazioni dietro ogni aggiornamento e fornendo ulteriori dettagli utili per la comprensione del progetto.

39

Grazie a queste funzionalità, un VCS elimina uno dei principali timori nello sviluppo software: la **paura di apportare modifiche** che potrebbero essere difficili da annullare o gestire.

L'utilizzo di un VCS diventa quindi essenziale per:

- Tracciare chi ha effettuato ciascuna modifica all'interno del team.
- Unire e integrare le modifiche di tutti i membri del gruppo.

Per raggiungere questo obiettivo, è necessario un punto centralizzato, accessibile a tutti, che collezioni e renda disponibili le modifiche, garantendo una collaborazione fluida e organizzata.

3.2 GIT

Come abbiamo già detto, Git è un VSC che mira a migliorare la qualità dell'esperienza di programmazione, mantenendo in memoria ogni versione del software che andiamo a creare. Le sue caratteristiche principali sono:

- **Velocità:** Le operazioni sono eseguite localmente, il che le rende più rapide rispetto al lavorare sempre su un server remoto.
- **Indipendenza:** Poiché Git lavora principalmente con copie locali, il server remoto è opzionale e utilizzato solo per condividere modifiche tra più sviluppatori.

Terminologia chiave:

- **Working Directory:** La directory contenente i file su cui si sta lavorando attivamente.
- **Repository:** L'archivio di tutte le versioni dei file e del progetto, solitamente non visibile direttamente allo sviluppatore.
- **Registrazione:** Il processo di creazione di una nuova versione di uno o più file

all'interno del repository.

40

È fondamentale capire che la Working Directory e il Repository sono entità separate, che non necessariamente coincidono. In aggiunta, Git utilizza un'area intermedia chiamata "**Stage**" o "Index", dove vengono raccolti i file pronti per essere registrati nel repository. Solo i file aggiunti allo Stage verranno effettivamente salvati nel Repository durante la registrazione.

L'area di lavoro è quindi divisa in tre:

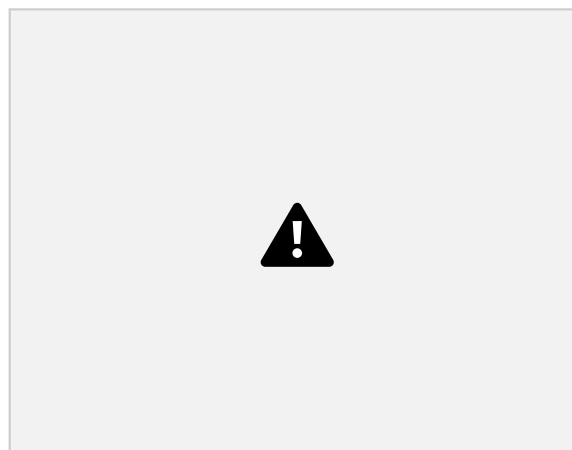
1. Working Directory
2. Stage
3. Repository

Ogni volta che viene effettuata una nuova registrazione (o **commit**), il VCS memorizza solo le differenze (o cambiamenti) rispetto all'ultima versione registrata dei file, a cui viene assegnato un identificativo univoco. Questo permette di tracciare ogni modifica. Ogni commit include i seguenti elementi:

- **Identificatore** univoco
- **Autore** della modifica
- **Data** del commit
- **Messaggio** descrittivo che spiega le modifiche

Per recuperare una versione precedente di un file o progetto, si utilizza il comando **checkout**, che consente di riportare la Working Directory a un punto specifico nella storia del repository.

Workflow di lavoro di Git in termini di stato dei file:



3.3 BITBUCKET

Bitbucket è un servizio online che fornisce soluzioni per la gestione di repository Git remoti. Sebbene sia un servizio a pagamento, offre un piano gratuito per progetti che coinvolgono fino a 5 utenti, rendendolo accessibile anche per piccoli team di sviluppo.

Ogni repository remoto su Bitbucket è identificato non solo dal nome, ma anche da un URL specifico nel formato:

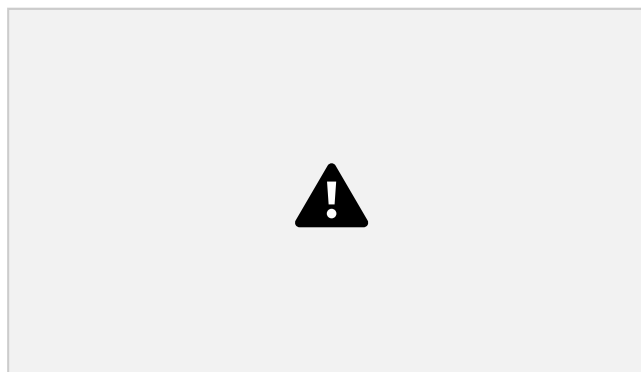
`https://nome-utente@bitbucket.org/nome-utente/nome-repository.git` In questo

URL:

- nome-utente rappresenta l'utente proprietario del repository o chi lo ha creato. •

nome-repository è il nome del repository stesso.

Schema generale per capire come funziona Bitbucket:



3.4 JSP

Il linguaggio server-side si riferisce a tecnologie in cui il codice della pagina web viene eseguito e interpretato direttamente sul server, riducendo significativamente il carico di lavoro sui dispositivi client. Un esempio di tecnologia server-side è **JSP (Java Server Pages)**, che, come suggerisce il nome, è strettamente legata al linguaggio Java.

JSP è una tecnologia semplice ma potente che consente di creare pagine HTML dinamiche lato server. Utilizza un insieme di tag speciali all'interno delle pagine HTML, che permettono di invocare funzioni predefinite tramite codice Java (JSTL) o JavaScript. Inoltre, JSP consente di creare librerie personalizzate di tag (JSP Custom Tag Library), estendendo così le funzionalità dei tag standard.

Una pagina JSP può essere vista come una rappresentazione ad alto livello di un servlet, ovvero un oggetto Java che opera all'interno di un server web per generare applicazioni web. A differenza dei servlet, JSP offre un vantaggio importante: la separazione tra la logica di gestione delle operazioni (business logic) e la parte di visualizzazione, facilitando la gestione del codice.

Una pagina JSP si compone di:

- **Codice statico:** Porzioni di HTML, CSS, JavaScript, ecc.
- **Codice dinamico:** Porzioni di codice Java, compilate dal motore JSP La pagina

JSP può contenere diversi tipi di elementi:

- **Direttive:** Forniscono informazioni globali al motore JSP (es. include, page).
- **Script JSP:** Frammenti di codice Java incorporati (dichiarazioni, espressioni, scriptlet).
- **Oggetti impliciti:** Parola chiave *this* per fare riferimento all'istanza corrente della pagina.
- **Azioni:** Incapsulano compiti comuni come la creazione o manipolazione di oggetti.

Come funzionano le JSP?

- Quando un client richiede una pagina JSP per la prima volta, il server compila la pagina creando un **servlet** corrispondente che rimarrà in memoria.
- Il server invia l'output, come pagina HTML, al browser del client.
- Alle richieste successive, se la pagina JSP non è stata modificata, il server riutilizza il servlet già compilato, altrimenti lo ricompila aggiornandolo.

!!La prima volta che si invoca una pagina JSP il sistema è un po' più lento del normale mentre durante le invocazioni successive si avranno prestazioni migliori.



JSP offre una serie di **vantaggi** significativi:

- **Prestazioni superiori:** Gli elementi JSP possono essere incorporati dinamicamente nelle pagine HTML, permettendo un'esecuzione più efficiente e rapida rispetto a molte altre tecnologie server-side.
- **Potenza e facilità d'uso:** JSP fornisce un approccio versatile, che permette di creare pagine web dinamiche sfruttando la potenza di Java in modo intuitivo. La sintassi semplice e l'integrazione con Java rendono lo sviluppo meno complesso.
- **Modifica semplificata delle pagine HTML:** Con JSP, è facile scrivere o modificare il codice HTML senza dover affrontare una complessità eccessiva di istruzioni tecniche, permettendo una gestione più fluida del contenuto dinamico.

In sintesi, JSP offre una soluzione versatile per la creazione di pagine web dinamiche, garantendo un'ottimizzazione delle prestazioni grazie alla gestione efficiente del codice sul server.



4.1 COS'È?

HTML, acronimo di **HyperText Markup Language**, è un linguaggio a marcatori, non un linguaggio di programmazione, poiché non soddisfa i tre criteri essenziali di un linguaggio di programmazione:

1. Sequenza
2. Ripetizione
3. Selezione

HTML si concentra sulla rappresentazione e visualizzazione delle informazioni, senza consentire logiche complesse come cicli o condizioni, tipiche di linguaggi di programmazione.

Nato per il web, HTML opera nel contesto del modello client-server, in cui due entità (client e server) si scambiano richieste e risposte. I server, spesso situati all'interno di **Data Center**, sono organizzati in armadi (rak) e devono essere in grado di servire più client contemporaneamente.

Un esempio di un altro linguaggio a marcatori è **XML**, che, a differenza di HTML, è orientato alla descrizione e struttura delle informazioni piuttosto che alla loro visualizzazione.

4.2 I TAG

I tag sono gli elementi fondamentali del linguaggio HTML. Sono parole chiave racchiuse tra

parentesi angolari (<...>), utilizzate generalmente in coppia per definire apertura e chiusura (ad esempio <p> e </p>).

I tag permettono di:

- **Formattare il testo**, attribuendogli una forma specifica.
- **Definire hyperlink**, ossia collegamenti ad altre pagine o risorse.
- **Raccogliere informazioni** dall'utente tramite moduli (form), che contengono campi in cui inserire dati.

Un documento HTML è strutturato ad albero, con i tag annidati in una gerarchia. Questo significa che ogni tag aperto deve essere chiuso prima di terminare il tag padre. Una corretta impaginazione e formattazione del codice non solo migliora la leggibilità, ma facilita anche la manutenzione e la revisione del codice stesso.

Tag di blocco e liste



Ogni tag HTML può contenere attributi, che servono a specificare meglio il comportamento del tag stesso. Gli attributi vengono indicati all'interno del tag di apertura seguendo questa struttura:

attributo="valore"

Ciascun tag supporta un insieme specifico di attributi. Ad esempio, il tag `<a>` utilizza l'attributo **href** per definire il collegamento:

```
<a href="http://www...">lo sono un link</a>
```

Di default, tutti i tag HTML occupano il 100% della larghezza del loro contenitore e un'altezza pari allo spazio necessario per mostrare il loro contenuto, a meno che non venga specificato diversamente tramite gli **attributi** width e height o con modifiche applicate tramite CSS.

Inoltre, i tag HTML tendono a posizionarsi uno sotto l'altro interrompendo la riga corrente, salvo alcune eccezioni specifiche.

PARAGRAFO

```
<p>Questo è un paragrafo</p>
```

Per andare a capo all'interno di un testo, si utilizza il tag `<br/>` . **TITOLI**

```
<h1>Questo è il titolo principale</h1>
```

```
<h2>Questo è un Sottotitolo</h2>
```

```
<h3>Questo è un Sottotitolo</h3>
```

```
<h4>Questo è un Sottotitolo</h4>
```

```
<h5>Questo è un Sottotitolo</h5>
```

```
<h6>Questo è un Sottotitolo</h6>
```

In una pagina HTML, dovrebbe essere presente un solo titolo `<h1>`, mentre i titoli da `<h2>` a `<h6>` possono essere utilizzati più volte. È fondamentale organizzare i

47

titoli in modo gerarchico per garantire una struttura chiara e coerente all'interno della pagina.

ANCORE

```
<a href="..." target="..." title="...">Questo è un link</a>
```

 Il tag `<a>` dispone di tre

attributi principali:

- **href**: indica l'indirizzo verso cui il link deve reindirizzare l'utente.
- **target**: definisce come deve avvenire l'apertura del collegamento (ad esempio, in una nuova scheda o nella stessa).
- **title**: specifica un testo descrittivo che viene mostrato come tooltip quando si passa il mouse sull'ancora.

CONTENITORI

`<div>`

`<p> ... </p>`

`<div> ... </div>`

`</div>`

Il tag `<div>` (abbreviazione di "Content Division") è usato per organizzare e suddividere la pagina in blocchi o sezioni logiche.

I tag contenitori come `<div>` spesso includono gli attributi `id` e `class`, che consentono di identificare il loro scopo e ruolo specifico, facilitando la gestione tramite CSS e JavaScript.

In HTML5, l'uso dei contenitori `<div>` è stato in parte sostituito da elementi semantici come `<section>`, `<nav>`, `<footer>`, e molti altri. Questi tag descrivono in modo più chiaro il ruolo e lo scopo delle diverse sezioni di una pagina, migliorando sia la leggibilità del codice che l'accessibilità.

48

Ad esempio:

- **<section>**: usato per raggruppare contenuti tematici o sezioni di una pagina.
- **<nav>**: definisce un'area destinata ai collegamenti di navigazione.
- **<footer>**:

identifica il piè di pagina di un documento o di una sezione.

- **<header>**: rappresenta l'intestazione di un documento o di una sezione, contenente

titoli, sottotitoli, loghi, o altri elementi introduttivi.

- **<main>**: indica il contenuto principale del documento, unico per ogni pagina, e esclude intestazioni, barre di navigazione, e piè di pagina.

Questi tag semantici rendono il markup più intuitivo e significativo rispetto all'uso generico di `<div>`.

LE LISTE

- ordinate `<ol>`
`<ol>`
`<li> ... </li>`
`<li> ... </li>`
`</ol>`
- non ordinate `<ul>`
`<ul>`
`<li> ... </li> <li> ... </li> </ul>`

I tag lista possono essere di 2 tipi:

La differenza è che di base, gli elementi di una lista ordinata sono preceduti da un numero progressivo, mentre gli elementi della lista non ordinata, da un bullet (•) chiamato “disc”.

IMMAGINI

`<img src=“...” alt=“...” />`

Il tag `<img>` viene utilizzato per inserire immagini in una pagina web. Ha due attributi principali:

49

- **src**: specifica il percorso dell'immagine, che può essere un file locale sul server o un URL sul web.
- **alt**: fornisce un testo descrittivo che viene mostrato se l'immagine non può essere caricata. È utile per migliorare l'accessibilità e il SEO.

ALTRI TAG

Il tag `<br/>` viene utilizzato per interrompere la riga di testo e andare a capo senza creare un nuovo paragrafo. Viene usato principalmente per gestire ritorni a capo all'interno di una stessa sequenza di testo.

Il tag `<hr/>` inserisce una linea orizzontale di separazione tra i contenuti, utile per dividere visivamente diverse sezioni di una pagina.

TAG IN LINEA

I **tag in linea** vengono definiti tali perché, a differenza di altri elementi, non interrompono il flusso della riga. In altre parole, gli elementi inline si dispongono uno accanto all'altro, senza forzare una rottura o un'interruzione nel layout del testo.

È importante sottolineare che tutti i tag inline devono essere contenuti all'interno di elementi di blocco o di liste. Questa struttura garantisce una corretta definizione del comportamento e della formattazione delle porzioni di contenuto, mantenendo l'integrità e la leggibilità del layout complessivo della pagina.

TAG `<HEAD>`

Il tag `<head>` rappresenta la sezione iniziale di un documento HTML, ed è fondamentale per definire i metadati e le risorse necessarie alla corretta interpretazione e presentazione della pagina. Pur non essendo visibile direttamente all'utente, il contenuto del `<head>` influisce su aspetti come il comportamento, lo stile e l'ottimizzazione SEO della pagina. Tra gli elementi tipicamente inclusi nel `<head>` troviamo:

50

- **`<title>`**: Specifica il titolo della pagina, che appare nella barra del titolo del browser e nei risultati dei motori di ricerca.
- **`<meta>`**: Utilizzato per definire metadati come il set di caratteri, la descrizione della pagina, parole chiave e altre informazioni rilevanti.
- **`<link>`**: Collega risorse esterne, come i fogli di stile CSS.
- **`<script>`**: Integra o collega script JavaScript.

- **<style>**: Consente di inserire regole di stile direttamente all'interno del documento.

TAG <META>

Il tag **<meta>** è un elemento chiave per la comunicazione di metadati all'interno di un documento HTML. Questi metadati, pur non essendo visibili sullo schermo, forniscono informazioni cruciali ai browser, ai motori di ricerca e ad altri agenti, contribuendo a migliorare la SEO e a garantire una corretta interpretazione del contenuto.

TAG <TABLE>

Il tag **<table>** è l'elemento principale per visualizzare dati in formato tabellare. Le tabelle sono strutturate per righe, dove ogni riga è definita dal tag **<tr>**, e le colonne all'interno di ogni riga sono rappresentate dal tag **<td>**. Questa struttura consente di organizzare e presentare informazioni in modo ordinato e facilmente leggibile.

TAG <THEAD>

Il tag **<thead>** definisce la testata della tabella. All'interno di questa sezione è possibile includere una o più righe di intestazione, solitamente composte dal tag **<tr>**. Per indicare le celle di intestazione, è consigliabile utilizzare il tag **<th>**, il quale enfatizza il contenuto e ne migliora la semantica, soprattutto in relazione all'accessibilità e alla strutturazione dei dati.

TAG <TBODY>

Il tag **<tbody>** rappresenta il corpo principale della tabella. In questa sezione vengono inserite tutte le righe che contengono i dati effettivi, organizzate mediante il tag **<tr>** per le righe e **<td>** per le celle. Separando il corpo dalla testata e dal footer, è possibile gestire e utilizzare in modo più preciso le differenti sezioni della tabella.

TAG <TFOOT>

Il tag **<tfoot>** è destinato a definire il footer della tabella. Analogamente al **<thead>**, questa sezione può contenere una o più righe (definite con **<tr>**), ognuna delle quali può includere celle di tipo **<td>**. Il footer è utile per includere informazioni riassuntive o note esplicative

relative ai dati contenuti nella tabella.

TAG <FORM>

Il tag `<form>` è fondamentale per costruire un modulo di raccolta dati che verranno inviati al server per essere elaborati, ad esempio, per il salvataggio in un database. Questo elemento HTML permette al browser di:

- **Raccogliere i dati** inseriti dall'utente attraverso vari controlli (come campi di testo, checkbox, pulsanti, ecc.);
- **Stabilire una connessione HTTP** con il server all'indirizzo specificato nell'attributo `action` e inviare i dati raccolti utilizzando il metodo definito nell'attributo `method`.

Il tag `<form>` possiede due attributi essenziali:

- **action**: Indica l'indirizzo della pagina o dello script server a cui inviare i dati del modulo. Questo URL rappresenta il punto di destinazione per l'elaborazione delle informazioni raccolte.
- **method**: Specifica il metodo (o verbo) HTTP da utilizzare per l'invio dei dati. I metodi più comuni sono:
 - **GET**: I dati vengono inviati tramite la query string dell'URL. Questo metodo è indicato per operazioni di recupero dati, ma non per inviare informazioni sensibili.
 - **POST**: I dati vengono inviati nel corpo della richiesta HTTP, garantendo una maggiore sicurezza e permettendo l'invio di una quantità maggiore di informazioni, come nel caso di dati riservati o complessi.

52

Ma come vengono effettivamente raccolti e gestiti i dati all'interno di un form?

Il linguaggio PHP mette a disposizione due array superglobali che contengono i dati ricevuti tramite la chiamata HTTP:

- **\$_GET[]**: Contiene i dati inviati utilizzando il metodo GET. Questi dati vengono trasmessi tramite la query string dell'URL;
- **\$_POST[]**: Contiene i dati inviati utilizzando il metodo POST. I dati vengono invece inclusi nel corpo della richiesta HTTP.

I dati sono accessibili all'interno di questi array attraverso le chiavi che corrispondono al

valore dell'attributo name degli elementi del form. Ad esempio, supponiamo di avere due campi nel modulo denominati nome e cognome; in PHP potremo recuperarli nel seguente modo:

```
$name = $_POST['nome'];
```

```
$surname = $_POST['cognome']
```

4.3 WEB SEMANTICO

Il Web semantico rappresenta uno degli obiettivi principali di HTML5, il quale introduce una serie di nuovi tag per migliorare la struttura e la comprensione dei contenuti, sia da parte degli utenti che delle macchine, come i motori di ricerca. Questi tag non sono pensati per modificare direttamente l'esperienza dell'utente finale, ma per rendere più chiaro e identificabile il significato delle varie sezioni della pagina, facilitando l'indicizzazione e l'interazione automatica con il contenuto. In altre parole, HTML5 non solo rende il web più accessibile, ma aiuta anche i motori di ricerca a comprendere meglio la struttura del sito, migliorando così la **visibilità** e la **gestione** dei contenuti.

NUOVI TAG SEMANTICI IN HTML5

Tra le principali innovazioni introdotte da HTML5, vi sono i tag semantici, che permettono di identificare e separare chiaramente le varie sezioni di una pagina web, migliorandone la

comprensione. Alcuni di questi tag includono:

- **<header>** e **<footer>**: Questi tag vengono utilizzati per definire rispettivamente le intestazioni e i piè di pagina di un documento o di una sezione. Forniscono un chiaro significato alla parte iniziale e finale di una pagina, rendendo la navigazione più intuitiva.
- **<aside>**: Utilizzato per creare contenuti laterali o sidebars, che non fanno parte del flusso principale del documento, ma che possono contenere informazioni correlate o accessorie, come una barra laterale con link utili, pubblicità o citazioni.
- **<menu>** e **<nav>**: Questi tag sono pensati per la gestione dei menu di navigazione. Il tag **<nav>** è usato per il menu di navigazione principale, mentre **<menu>** viene utilizzato per altri tipi di menu, come quelli contestuali o per azioni di interazione.

54

- **<section>** e **<article>**: Questi tag sono fondamentali per organizzare il contenuto in sezioni tematiche e articoli autonomi e ripetibili all'interno della pagina. L'uso di **<section>** aiuta a suddividere il contenuto in blocchi tematici, mentre **<article>** viene impiegato per contenuti che possono essere distribuiti autonomamente, come post di blog o notizie.

VANTAGGI DEL WEB SEMANTICO

L'adozione di questi tag semantici permette di dare un significato più chiaro alla struttura di una pagina web, migliorando la sua interpretazione da parte di motori di ricerca, applicazioni automatizzate e strumenti di accessibilità. L'integrazione di questi marcatori aiuta a migliorare l'indicizzazione del contenuto, semplificando la ricerca di informazioni da parte degli utenti. In questo modo, si ottimizza non solo l'accessibilità del sito, ma anche la user experience (esperienza dell'utente finale), rendendo la navigazione più fluida, intuitiva e pertinente.



In sintesi, **HTML5** non solo arricchisce il web dal punto di vista delle funzionalità interattive, ma soprattutto contribuisce a costruire un web **semantico**, dove il significato del contenuto è chiaro non solo agli utenti, ma anche alle macchine che gestiscono e indicizzano le informazioni. Questo approccio facilita la costruzione di siti web più interattivi, sicuri, accessibili e facilmente fruibili, rispondendo meglio alle esigenze dell'utente moderno.

55

5.1 COS'È?



Per fogli di stile si intende il codice **CSS** (**C**ascading **S**tyle **S**heets, ovvero fogli di stile a cascata), un linguaggio utilizzato per definire la formattazione dei documenti HTML, XHTML e XML. Il termine "a cascata" deriva dal fatto che, pur potendo essere applicati molteplici fogli di stile, le regole vengono ereditate in cascata, consentendo a un foglio di stile di integrare e sovrascrivere le proprietà dei precedenti.

Le direttive che regolano e compongono il linguaggio CSS sono stabilite dal W3C, garantendo standard e uniformità nell'uso del linguaggio. Il principale obiettivo dell'introduzione del CSS è stato quello di separare il contenuto di una pagina web dalla sua presentazione. Questa separazione non solo rende il codice più chiaro e manutenibile, ma permette anche il riutilizzo di porzioni di codice, evitando la duplicazione delle regole di formattazione.

In sintesi, grazie al CSS è possibile definire in modo centralizzato e modulare le regole che determinano come le informazioni verranno visualizzate sul sito, mantenendo distinti gli

aspetti relativi alla struttura e al contenuto da quelli inerenti allo stile, come font, colori, dimensioni, impaginazione e gestione dei titoli.

56

5.2 INTEGRARE CODICE CSS

Esistono tre diversi modi per integrare il codice CSS all'interno di una pagina web: fogli di stile esterni, stili interni e stili in linea (inline). Ognuna di queste modalità ha vantaggi e svantaggi a seconda delle esigenze di sviluppo.

FOGLI DI STILE ESTERNI

Il metodo più consigliato è quello di collegare un file CSS esterno alla pagina HTML. Questo si realizza inserendo nel tag `<head>` un collegamento al foglio di stile tramite il tag `<link>`.

✓ Vantaggi:

- Permette di riutilizzare lo stesso file CSS su più pagine, migliorando l'efficienza e la manutenzione del codice.
- Rende il codice HTML più pulito e leggibile, separando contenuto e presentazione.
- Facilita l'aggiornamento globale dello stile, poiché una modifica al file CSS si riflette su tutte le pagine che lo utilizzano.

✗ Svantaggi:

Richiede una richiesta HTTP aggiuntiva per caricare il file, il che può leggermente

influenzare le prestazioni.

STILI INTERNI (CSS NELL'HEAD DELLA PAGINA)

È possibile definire le regole CSS all'interno dello stesso file HTML utilizzando il tag `<style>` nella sezione `<head>`.

✓ Vantaggi:

- Utile quando si deve applicare uno stile specifico a una singola pagina, senza influenzare altre pagine del sito.
- Non richiede una richiesta HTTP aggiuntiva come i fogli esterni. 57

✗ Svantaggi:

- Minor riuso del codice, poiché le regole CSS sono applicabili solo a quella specifica pagina.
- Meno organizzato, rendendo più difficile la gestione dello stile su progetti di grandi dimensioni.

STILI IN LINEA (INLINE CSS)

CSS può essere applicato direttamente all'interno degli elementi HTML utilizzando l'attributo `style`.

✓ Vantaggi:

Utile per modifiche rapide o per applicare uno stile specifico a un singolo elemento senza creare un nuovo file CSS.

✗ Svantaggi:

- Difficile da mantenere su progetti complessi, perché il codice CSS si mescola con l'HTML.
- Mancanza di riuso: lo stile deve essere ripetuto ogni volta che serve, aumentando la ridondanza.
- Prevalenza sugli altri stili: gli stili inline sovrascrivono sia quelli interni che quelli esterni, il che può creare problemi di gestione.

5.3 PRIORITÀ TRA GLI STILI

In generale, la gerarchia di applicazione degli stili segue questa regola: • **Stili**

in linea (inline) → hanno la precedenza assoluta.

- **Stili interni** (nel `<head>`) → sovrascrivono quelli esterni ma non quelli in linea.
- **Fogli di stile esterni** → vengono applicati solo se non esistono regole più specifiche.

58

Per ottenere un codice più pulito e scalabile, l'uso dei fogli di stile esterni è la soluzione preferibile nella maggior parte dei casi.

5.4 STRUTTURA DI UN FOGLIO DI STILE CSS

Un foglio di stile CSS è composto da una sequenza di regole che definiscono lo stile degli elementi HTML. Ogni regola si compone di due parti principali:

- 1) Il **selettore**, che identifica gli elementi a cui applicare lo stile.
- 2) Il **blocco di dichiarazioni**, racchiuso tra parentesi graffe `{ }`, contenente una o più dichiarazioni.

SELETTORI CSS

I **selettori** permettono di individuare specifici elementi HTML a cui applicare le regole di stile. Alcuni tipi di selettori comuni includono:

- **Nome tag** → seleziona tutti gli elementi con quel nome (es. `h1`, `p`, `a`).
- **Classi** (`.nome-classe`) → seleziona tutti gli elementi con una determinata classe.
- **Identificatori** (`#nome-id`) → seleziona un elemento con un identificatore univoco.

BLOCCHI DI DICHIARAZIONE

Ogni **blocco di dichiarazione** contiene una o più dichiarazioni, ognuna delle quali è composta da:

- Una proprietà → specifica quale aspetto modificare (es. color, font-size).
- Un valore → assegna un valore alla proprietà (es. blue, 16px).

Le proprietà e i valori sono separati dai due punti (:) e ogni dichiarazione termina con un punto e virgola (;).

59

5.5 GERARCHIA NEL CODICE CSS

Il termine "**Cascading**" in CSS si riferisce al modo in cui le regole di stile vengono applicate quando ci sono conflitti. In altre parole, se più istruzioni definiscono lo stesso stile per un elemento, il browser segue una precisa gerarchia per determinare quale regola ha la priorità.

La regola più **specificata** ha sempre la precedenza su quelle più generiche. Ad esempio, se una regola si applica a tutti i paragrafi di una pagina e un'altra si applica solo a una determinata classe di paragrafi, sarà quest'ultima ad essere seguita.

Quando due regole hanno lo stesso livello di specificità, viene applicata quella scritta più in basso nel foglio di stile. Questo significa che, se un elemento ha due regole in conflitto, l'ultima dichiarata avrà la priorità.

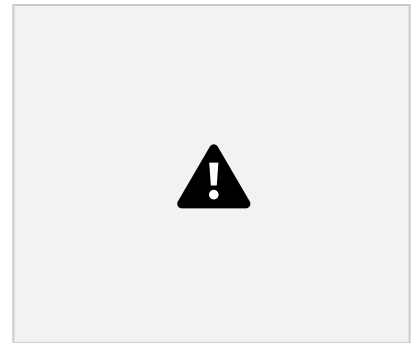
Un altro aspetto da considerare è l'ordine con cui vengono caricati i fogli di stile. Le regole scritte direttamente all'interno dell'attributo style di un elemento HTML hanno la priorità su quelle definite in un foglio di stile interno alla pagina, mentre queste ultime prevalgono su quelle contenute in un file CSS esterno.

Per forzare una regola e fare in modo che venga applicata indipendentemente da qualsiasi altro criterio, si può utilizzare il modificatore **!important**. Tuttavia, il suo utilizzo andrebbe limitato ai soli casi necessari, poiché potrebbe complicare la gestione e la modifica del codice CSS.

5.6 MARGINI E PADDING

Nel linguaggio CSS, margini e padding sono fondamentali per la gestione degli spazi tra gli elementi e la loro disposizione all'interno di una pagina web.

- Il **Margine** (margin) determina lo spazio esterno che separa un elemento dagli altri contenuti della pagina.
- Il **Padding**, invece, definisce lo spazio interno tra il contenuto di un elemento e il suo bordo.



60

Entrambi possono essere impostati singolarmente per i quattro lati di un elemento o in modo uniforme su tutti i lati.

5.7 BOOTSTRAP

Bootstrap è un framework front-end open-source che facilita la creazione di interfacce web moderne e responsive. Sviluppato inizialmente da Twitter, è oggi uno degli strumenti più utilizzati per la progettazione di siti e applicazioni web grazie alla sua semplicità e versatilità.

CARATTERISTICHE PRINCIPALI

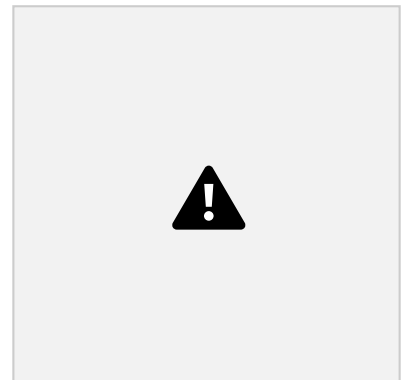
- **Sistema a Griglia** (Grid System): Bootstrap utilizza un sistema di griglia flessibile e adattabile basato su colonne, che permette di creare layout responsive in modo intuitivo. Questo sistema è fondamentale per garantire una corretta visualizzazione su dispositivi di diverse dimensioni, dal desktop agli smartphone.
- **Componenti Predefiniti**: Il framework include numerosi componenti già stilizzati e pronti all'uso, come pulsanti, modali, card, navbar e tabelle, riducendo significativamente il tempo di sviluppo.
- **Compatibilità e Responsività**: Bootstrap è progettato per essere compatibile con tutti i browser moderni e garantisce un adattamento automatico dei contenuti in base alle dimensioni dello schermo.
- **Personalizzazione**: Sebbene offra uno stile predefinito, Bootstrap permette un'ampia personalizzazione grazie all'uso di variabili CSS e alla possibilità di sovrascrivere le classi con fogli di stile personalizzati.
- **Utilizzo di JavaScript**: Il framework include anche una serie di componenti interattivi basati su JavaScript, come slider, popover e collapsed, che arricchiscono

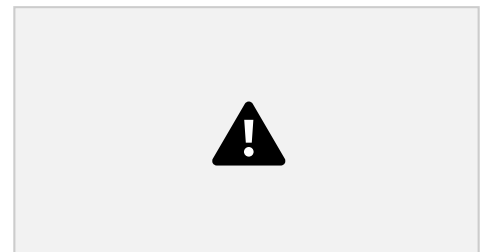
l'esperienza utente senza la necessità di scrivere codice complesso.

61

ELEMENTI FONDAMENTALI

- **Container:** È l'elemento che racchiude il contenuto della pagina e ne gestisce la larghezza in modo responsive. Può essere fluido (adatta la larghezza a tutto lo schermo) o con una dimensione fissa in base al breakpoint dello schermo.
- **Row:** È la riga all'interno di un container che organizza le colonne. Ogni row è suddivisa in 12 parti uguali, consentendo la disposizione flessibile degli elementi.
- **Column:** Le colonne definiscono la larghezza degli elementi all'interno di una riga. Possono avere diverse dimensioni in base al dispositivo (es. col-6 per occupare metà della riga su tutti gli schermi, col-md-4 per occupare 4 parti su 12 solo su schermi medi e più grandi).
- **Classi Hidden e Visible**
 - **Hidden:** Permette di nascondere un elemento su specifiche dimensioni dello schermo (es. d-none, d-md-none).
 - **Visible:** Viene gestito tramite classi display (d-block, d-flex) per mostrare un elemento solo in determinati breakpoint.





6.1 COS'È?

PHP (*PHP: Hypertext Preprocessor* o *Personal Home*

Page Tools), è un **linguaggio di scripting** lato server largamente utilizzato nello sviluppo web. La sua sintassi è simile a quella del linguaggio C, rendendolo intuitivo per chi ha già familiarità con altri linguaggi di programmazione.

Negli anni, PHP ha subito numerose evoluzioni, migliorando sia in termini di prestazioni che di funzionalità. In particolare, a partire dalla versione 3.0, è stato introdotto il supporto alla programmazione orientata agli oggetti, mentre con PHP 7.0 sono stati apportati notevoli miglioramenti alle prestazioni, rendendolo fino a due volte più veloce rispetto alle versioni precedenti.

PHP è un **linguaggio interpretato**, il che significa che il codice non viene compilato in un file eseguibile, ma eseguito direttamente da un interprete. Questo consente una maggiore flessibilità nello sviluppo e nel debugging del codice, ma lo rende generalmente meno efficiente rispetto ai linguaggi compilati.

Un'altra caratteristica è l'utilizzo della **tipizzazione dinamica**, ciò significa che non è necessario specificare esplicitamente il tipo di una variabile. Il linguaggio assegna automaticamente il tipo in base al valore assegnato alla variabile. Tuttavia, dalla versione 8.0,

è stato introdotto un maggiore controllo sui tipi, permettendo di dichiarare esplicitamente i tipi di variabili, parametri e valori di ritorno delle funzioni.

Per inserire il codice PHP in HTML basta:

1. Nominare il file come `.php` e non come `.html`
2. Inserire il codice PHP fra i tag `<?php` e `?>`

63

6.2 LE CLASSI

Le **classi** sono l'entità fondamentale della programmazione orientata agli oggetti (OOP). Ci permettono di definire tipi di dato complessi, modellando concetti del mondo reale attraverso la combinazione di proprietà (caratteristiche) e metodi (comportamenti).

Possiamo pensare alle classi come a delle *struct* potenziate rispetto a quelle presenti in linguaggi come il C.

<pre>struct Person { char *f_name; char *l_name; }</pre>	<pre>class Person { public \$first_name; public \$last_name; }</pre>
---	---

Per creare una nuova istanza della classe, ovvero un oggetto che possiede tutte le proprietà definite nella classe stessa, utilizziamo la parola chiave “*new*”.

```
$p = new Person();
```

Le variabili definite all'interno di una classe sono chiamate **proprietà**, più correttamente **property** (o **attributi**). In C, questi elementi sono noti come campi. Per accedere a una proprietà di un oggetto, utilizziamo l'operatore freccia `->`.

```
$p->first_name = "Mario";  
echo $p->first_name;
```

6.3 I METODI

Oltre alle proprietà, le classi possono contenere **metodi**, ovvero funzioni che operano sugli oggetti della classe stessa; essi permettono di interagire con le proprietà e di definire il comportamento dell'oggetto.

All'interno di una classe, per riferirsi all'oggetto su cui stiamo operando, utilizziamo la variabile speciale *\$this*. Questa variabile rappresenta l'istanza corrente della classe e consente di accedere alle proprietà e ai metodi dell'oggetto.

64

```
class Persona {  
    public $nome;  
    public function saluta() {  
        return "Ciao, mi chiamo " . $this->nome;  
    }  
} ...
```

METODO COSTRUTTORE

Per assegnare un valore iniziale alle proprietà di un oggetto, si utilizza un metodo speciale chiamato **costruttore**. Il costruttore ha il compito di inizializzare le proprietà in modo corretto e controllato, evitando di doverle reimpostare manualmente dopo la creazione dell'oggetto. In PHP, il costruttore si definisce con il metodo speciale `__construct()`.

METODI GETTER E SETTER

L'accesso diretto alle proprietà di un oggetto non è consigliato per motivi di sicurezza e controllo dei dati. Per questo motivo, è buona pratica rendere le proprietà private e fornire metodi dedicati per ottenerne o modificarne i valori. Questi metodi sono noti come getter e setter.

- **Getter:** permette di leggere il valore di una proprietà.
- **Setter:** permette di modificare il valore di una proprietà, garantendo che vengano rispettate eventuali condizioni o vincoli.

✓ Vantaggi:

- **Incapsulamento**: proteggono i dati da modifiche non controllate.
- **Validazione**: i setter permettono di controllare i dati prima di assegnarli.
- **Flessibilità**: consentono di modificare la logica di accesso ai dati senza cambiare la struttura della classe.

65

6.4 MODIFICATORI

PHP utilizza i **modificatori di accesso** per controllare la visibilità delle proprietà e dei metodi all'interno della classe. I principali modificatori sono:

- **PUBLIC**: la proprietà o il metodo è accessibile da qualsiasi parte del codice (per esempio i metodi).
- **PRIVATE**: l'accesso è limitato solo ai metodi definiti all'interno della stessa classe (per esempio le property).
- **PROTECTED**: l'accesso è consentito solo alla classe e alle sue eventuali sottoclassi.

!!STATIC è un **modificatore di stato**, viene utilizzato per dichiarare proprietà e metodi che appartengono alla classe stessa, piuttosto che a una sua istanza. Questo significa che: le proprietà statiche hanno lo stesso valore per tutte le istanze della classe e i metodi statici possono essere chiamati senza creare un'istanza della classe. Per esempio:

```
class myclass{  
    static public function mymethod(){...}  
}  
  
echo myclass::mtmethod();
```

6.5 OBJECT ORIENTED PROGRAMMING

La **Programmazione Orientata agli Oggetti (OOP)** è un paradigma di

programmazione che permette di organizzare il codice in **oggetti**, che interagiscono tra loro attraverso lo scambio di messaggi (metodi). Questo approccio consente di modellare in modo naturale il mondo reale o sistemi astratti, facilitando la gestione e la manutenzione del software.

L'OOP offre  vantaggi, come:

- **Migliore organizzazione del codice:** il codice è strutturato in classi, favorendo la modularità e il riuso.

66

- **Facilità di manutenzione:** il codice è più leggibile e scalabile.
- **Stratificazione e specializzazione:** consente la creazione di strutture gerarchiche tra classi.
- **Maggiore sicurezza:** grazie all'incapsulamento, è possibile proteggere i dati sensibili da accessi indesiderati.

L'OOP si basa su 3 concetti fondamentali:

1. **EREDITARIETÀ:** l'ereditarietà consente di creare gerarchie tra classi, permettendo a una classe figlia di riutilizzare e specializzare le funzionalità di una classe padre. Questo principio segue il concetto DRY (*Don't Repeat Yourself*), che evita la duplicazione del codice. L'ereditarietà è utile quando vogliamo modellare relazioni di tipo "è un", creando strutture più flessibili e riutilizzabili.
2. **INCAPSULAMENTO:** l'incapsulamento consente di organizzare e raggruppare i dati all'interno di una classe e di controllare l'accesso ai dati attraverso modificatori di accesso (public, private, protected). Serve a: nascondere i dettagli interni dell'implementazione di una classe, proteggere i dati da modifiche non autorizzate, fornire un'interfaccia pubblica per interagire con l'oggetto.
3. **POLIMORFISMO:** il polimorfismo permette di definire metodi con lo stesso nome, ma con comportamenti diversi, consentendo di adattare il comportamento delle classi in modo flessibile. Uno degli esempi più comuni di polimorfismo è l'*overriding*, che permette a una sottoclasse di ridefinire un metodo della classe padre.



6.6 PHP E HTML MIXING

Uno degli aspetti più importanti della programmazione in PHP è la sua integrazione con HTML. PHP è un **linguaggio lato server**, il che significa che il codice viene eseguito prima che la pagina venga inviata al browser dell'utente. Quando un file contenente sia PHP che HTML viene elaborato, il server prima esegue il codice PHP attraverso l'interprete e poi restituisce l'output HTML risultante al client.

Per poter eseguire codice PHP all'interno di una pagina web, è necessario che il file abbia estensione **.php** anziché **.html**; inoltre il codice deve essere inserito all'interno dei tag **<?php ?>**, che possono essere utilizzati più volte all'interno del file, per esempio:

```
<?php $titolo = "Benvenuto nel mio sito"; ?>
<h1><?php echo $titolo; ?></h1>
```

Quando si mescola PHP con HTML, è fondamentale adottare buone pratiche di scrittura del codice per garantire leggibilità e manutenibilità:

✗ SBAGLIATO:

```
<?php $titolo = "Titolo"; echo
"<h1>".$titolo."</h1>";?>
```

✓ CORRETTO:

```
<?php $titolo = "Titolo"; ?> <h1><?php
echo $titolo; ?></h1>
```

Quando si utilizzano cicli o condizioni in un contesto HTML, è preferibile utilizzare le forme alternative **for ... endfor**, **if ... endif**, **while ... endwhile**, piuttosto che le parentesi {...}.

L'uso di queste forme migliora la leggibilità, specialmente quando si scrive codice PHP in mezzo a tanto HTML.

```
<ul>
```

```
<?php for( ... ) : ?> <li> ... </li>
```

```
<?php endfor; ?> </ul>
```

```
<?php if( ... ) : ?> <p> ... </p>
```

```
<?php else: ?>
```

```
<p> ... </p>
```

```
<?php endif; ?> 68
```

6.7 PHP7

PHP 7 ha introdotto una serie di miglioramenti significativi rispetto alle versioni precedenti, tra cui un incremento delle prestazioni, una gestione della memoria più efficiente e l'aggiunta di nuove funzionalità per rendere il linguaggio più moderno e robusto.

Tra le principali novità troviamo:

- **OPERATORE SPACESHIP (<=>):** operatore di comparazione che permette di confrontare due valori; utile per ordinare array o confrontare valori in modo compatto. Restituisce: “-1” se il primo operando è minore del secondo, “0” se i due operandi sono uguali oppure “+1” se il primo operando è maggiore del secondo.
- **OPERATORE NULL COALESCING (??):** semplifica il controllo dell'esistenza di variabili o chiavi negli array associativi. Restituisce il primo operando se non è null, altrimenti restituisce il secondo.

```
$name = $_POST['nome'] ?? 'Anonimo';
```

- **DICHIARAZIONE DEL TIPO DI RITORNO:** prima di PHP 7, le funzioni in PHP potevano restituire qualsiasi tipo di dato senza alcun vincolo. Ora è possibile specificare il tipo di dato che una funzione deve restituire, aumentando la sicurezza e la leggibilità del codice. Se la funzione restituisce un valore di un tipo diverso da quello dichiarato, PHP genera un errore.

```
function getUser(): User {  
    return new User();  
}
```

```
}
```

- **DICHIARAZIONE DEL TIPO PER I PARAMETRI:** anche i parametri delle funzioni possono avere un tipo dichiarato. Se viene passato un valore di un tipo non compatibile, PHP tenta di convertirlo (modalità default) o genera un errore (modalità strict).

69

```
function setName(string $name) {  
  
    echo "Il tuo nome è: $name";  
  
}
```

- **MODALITÀ STRICT E DEFAULT:** di default PHP cerca di convertire automaticamente il valore passato al tipo richiesto; se vogliamo evitare conversioni indesiderate possiamo aggiungere all'inizio del file PHP:

```
declare(strict_types=1);
```

- **FUNZIONI ANONIME E CALLBACK:** le funzioni anonime sono funzioni senza nome che possono essere assegnate a variabili o passate come argomenti ad altre funzioni. Sono molto utili per la programmazione funzionale e le callback.

6.8 PHP8

PHP 8 ha introdotto numerosi miglioramenti e nuove funzionalità, tra cui un supporto più avanzato alla programmazione orientata agli oggetti. Tra le innovazioni più significative troviamo il named arguments, ovvero la possibilità di specificare i parametri per nome, indipendentemente dall'ordine in cui vengono passati; questa novità presenta dei vantaggi, tra cui: maggiore leggibilità, flessibilità e maggiore sicurezza.

```
function saluta($nome, $cognome, $eta) {  
    echo "Ciao, mi chiamo $nome $cognome e ho $eta anni."; }  
  
// Chiamata con named arguments (ordine diverso rispetto alla definizione)  
saluta(eta: 25, nome: "Marco", cognome: "Rossi");
```

6.9 FUNZIONI ANONIME

I moderni linguaggi di programmazione fanno uso delle funzioni anonime, note anche come **closures**.

Una funzione anonima è una funzione che non ha un nome esplicito, ma possiede parametri e codice eseguibile. A differenza delle funzioni tradizionali, che vengono definite con un nome specifico, le funzioni anonime possono:

- Essere assegnate a variabili e riutilizzate in diversi punti del codice.
- Essere passate come parametri ad altre funzioni o metodi.
- Essere utilizzate nelle callback, ovvero blocchi di codice che vengono eseguiti dopo il completamento di un'operazione.

```
class Esempio {  
    public function esegui($callback) {  
        echo "Prima dell'esecuzione della callback...\n";  
        $callback();  
        echo "Dopo l'esecuzione della callback...\n";  
    }  
}  
  
$oggetto = new Esempio();  
$oggetto->esegui(function() {  
    echo "Sto eseguendo la funzione anonima!\n";  
});
```

OUTPUT:

Prima dell'esecuzione della callback...

Sto eseguendo la funzione anonima!

Dopo l'esecuzione della callback...

71

La **Human-Computer Interaction** (HCI) è il campo di studio che analizza il modo in cui gli esseri umani interagiscono con i computer e i sistemi digitali. Il suo obiettivo è progettare interfacce intuitive, accessibili ed efficienti, migliorando l'**esperienza utente** (UX) attraverso principi di usabilità, design e psicologia cognitiva.

L'**HCI** è fondamentale nello sviluppo di applicazioni, siti web e dispositivi digitali, poiché un'interazione ben progettata può rendere la tecnologia più comprensibile e fruibile. Questo campo integra discipline come l'informatica, il design e le neuroscienze per creare sistemi che rispondano alle esigenze e ai comportamenti degli utenti.

In questa unità esploreremo i principi chiave dell'HCI, principi di UI design e interaction design.



72

1.1 IL FATTORE UMANO

Quando si progetta un sistema informatico, è



impossibile realizzare un'interfaccia che sia perfetta per tutti gli utenti. Le persone sono diverse sotto molti aspetti: fisici, cognitivi, culturali e sociali. Alcune di queste differenze possono influenzare in modo significativo l'interazione con il sistema. Per questo motivo, i progettisti selezionano un sottoinsieme di utenti target, cercando di comprendere le loro caratteristiche e i fattori che ne influenzano il comportamento, così da progettare interfacce accessibili e usabili, che garantiscono una buona esperienza utente al maggior numero possibile di persone. Tuttavia, è importante considerare che ai confini tra due sottoinsiemi il sistema potrebbe funzionare male o addirittura non funzionare affatto.

Un altro aspetto fondamentale nell'interazione uomo-computer è il **fattore tempo**. La conoscenza di tempistiche aiuta a progettare interfacce che rispettino i tempi naturali dell'utente, migliorando l'usabilità del sistema. Per analizzare queste tempistiche, Newell (1990) ha introdotto il modello della **Scala del Tempo Umano**, che suddivide le azioni in 4 bande temporali:

1. **Biologica** (da millisecondi a secondi): include i processi fisiologici e neurologici (es. riflessi, tempi di reazione).
2. **Cognitiva** (da centinaia di millisecondi a minuti): riguarda i processi mentali legati alla percezione e all'elaborazione delle informazioni (es. riconoscere un'icona, leggere un testo, prendere una decisione semplice).
3. **Razionale** (da minuti a ore): coinvolge attività che richiedono ragionamento e pianificazione (es. risolvere un problema, scrivere un documento).
4. **Sociale** (da ore a mesi o anni): riguarda le interazioni e i comportamenti influenzati dal contesto sociale e culturale (es. apprendimento di nuove tecnologie, evoluzione delle abitudini di utilizzo).

1.2 IL MODELLO RGB

La **vista** è uno dei sensi più importanti nell'interazione tra esseri umani e ambiente. L'organo deputato alla percezione visiva è l'**occhio**, che riceve la luce riflessa o emessa dagli oggetti circostanti e la converte in impulsi elettrici, successivamente interpretati dal cervello.

La **luce** è una forma di energia elettromagnetica, e la proprietà che guida la percezione del

colore è la **frequenza**, misurata in nanometri (nm). L'occhio umano è in grado di percepire solo una porzione limitata dello spettro elettromagnetico, chiamata **spettro visibile**, che si estende approssimativamente tra i 350 nm e i 750 nm.

Ogni lunghezza d'onda (o frequenza) è associata a un colore percepito:



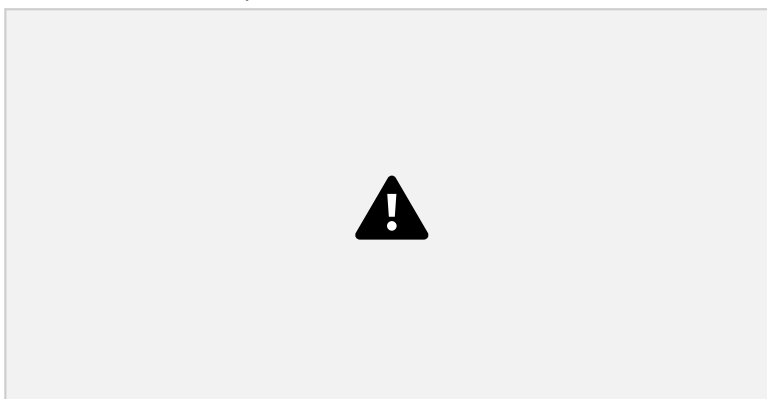
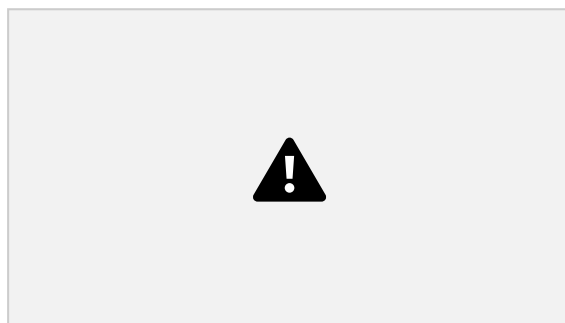
- **CORTA** (vicina ai 400 nm): associata a colori come **violetto** e **blu** •

INTERMEDIA (circa 500–600 nm): associata al **verde** e **giallo** • **LUNGA** (oltre

i 600 nm): associata a colori come **arancio** e **rosso**

Il **modello RGB** è uno standard internazionale per la rappresentazione dei colori nei contesti digitali, definito dalla Commission Internationale de l'Éclairage (CIE) nel 1931. L'acronimo RGB deriva dalle iniziali dei tre colori fondamentali su cui si basa: **RED**, **GREEN**, **BLUE**.

A differenza di quanto si pensi, questi non coincidono necessariamente con i colori primari tradizionali, bensì sono i colori fondamentali per la **sintesi additiva**, ovvero il processo attraverso cui si creano nuovi colori combinando le luci (per esempio: se tutti e tre i canali sono impostati al massimo livello (255), si otterrà il bianco; per contro, si otterrà il nero se si impostano tutti i canali a 0).



74

1.3 RGB IN CSS

Nel linguaggio CSS, il modello RGB è utilizzato per definire i colori in vari contesti visivi delle interfacce utente. Le principali proprietà CSS che supportano l'uso di colori RGB sono:

- colore del testo → `color: rgb(255, 0, 0);`

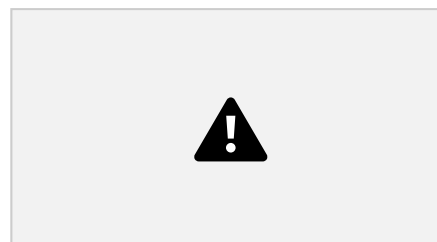
- colore di sfondo → `background-color: rgb(240, 240, 240);` • colore dei bordi

→ `border-color: rgb(230, 40, 120);`

Inoltre, CSS supporta anche una variante con un quarto parametro opzionale per la **trasparenza**, tramite la funzione `rgba()`, dove **a** sta per **alpha**, il canale dell'opacità (valore da 0.0 a 1.0) → `background-color: rgba(255, 0, 0, 0.5)`.

1.4 IL MODELLO HEX

Il **modello HEX** rappresenta i colori tramite una notazione esadecimale (base 16, da 00 a FF). Ogni colore è definito da una stringa di sei cifre precedute da #, dove le prime due rappresentano il rosso, le successive il verde e le ultime il blu (es. #FF0000 corrisponde al rosso puro).



1.5 I COLORI E IL LORO SIGNIFICATO

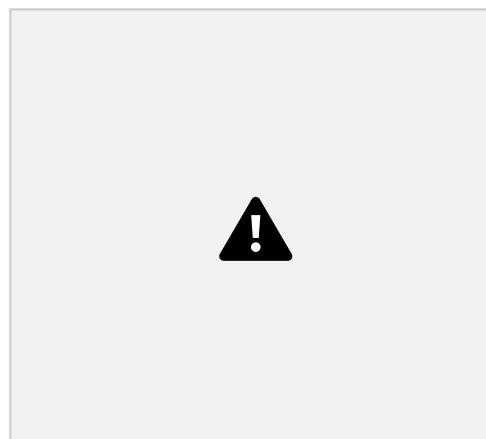
I colori non sono solo una scelta estetica: hanno un ruolo cruciale nel guidare l'attenzione, comunicare significato e trasmettere emozioni. Nel design di un'interfaccia, ogni colore può influenzare il comportamento dell'utente e l'efficacia dell'interazione. Ad esempio il rosso può indicare pericolo, errore, urgenza, il verde è spesso associato a conferme, successi, operazioni sicure, il giallo richiama l'attenzione o segnala un avviso, oppure il blu comunica affidabilità, calma e professionalità.

75

Utilizzare i colori in modo coerente e accessibile aiuta l'utente a comprendere lo stato del sistema, a navigare con sicurezza e a prendere decisioni più rapide. Inoltre, una corretta scelta cromatica deve tener conto anche di aspetti culturali e di accessibilità visiva.

1.6 LA LETTERATURA E YARBUS

Alfred Yarbus, studioso russo degli anni '60, è noto per i suoi esperimenti sull'**osservazione visiva**. Utilizzando uno strumento per tracciare i movimenti oculari, dimostrò che il modo in cui le



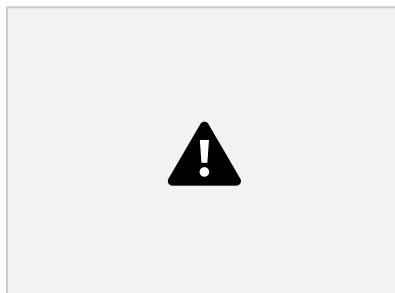
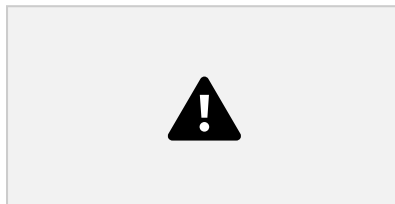
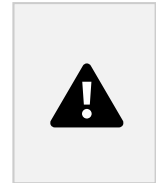
persone guardano un'immagine dipende
fortemente dallo scopo con cui la osservano.

Queste scoperte sono fondamentali per l'HCI e l'UI

Design: se sappiamo cosa sta cercando l'utente, possiamo guidare il suo sguardo e ottimizzare la disposizione dei contenuti scegliendo aree che attirano più facilmente l'attenzione.

1.7 HARD E SOFT CONTROLS

In origine, i sistemi interattivi si basavano su **Hard Controls**: pulsanti, leve, manopole e interruttori fisici, ognuno progettato per svolgere una sola funzione fissa. Una volta costruiti, non potevano essere modificati: il loro comportamento era vincolato alla loro forma e alla loro struttura meccanica.



Con l'arrivo dei computer e delle interfacce grafiche (GUI), sono nati i **Soft Controls**. A differenza di quelli fisici, questi non sono legati a una funzione permanente: il loro comportamento può cambiare dinamicamente a seconda del contesto o dello stato del sistema, offrendo flessibilità, adattabilità e un'interazione più ricca e immediata per l'utente. Quest'ultimi possono anche

76

funzionare da indicatori visivi (i pulsanti delle toolbar che si illuminano o cambiano stato quando una funzione è attiva, o gli slider che comunicano visivamente la proporzione tra l'area visibile e il contenuto totale, oltre che permettere di scorrere il contenuto).

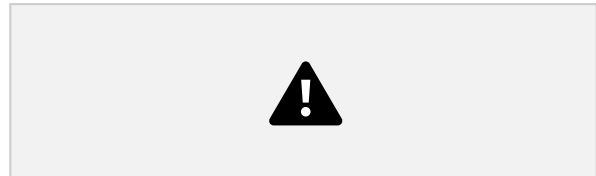
2.8 RELAZIONI CONTROLLER-DISPLAY

Le **relazioni controller-display** descrivono il legame tra un **dispositivo di input fisico** (controller, come il mouse) e la **risposta sullo schermo** (display, come il movimento del puntatore o della vista). Un'interazione efficace e intuitiva si basa su relazioni spaziali coerenti: quando muoviamo il mouse verso destra, ci aspettiamo che il puntatore si muova

nella stessa direzione. Questo comportamento è naturale, immediato e non richiede riflessione: è ciò che si definisce **compatibilità spaziale**.

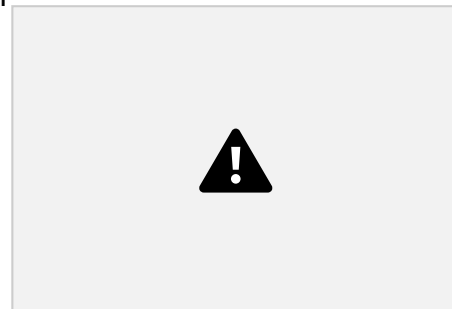
Tuttavia, alcune relazioni spaziali non sono innate, ma devono essere apprese. Ad esempio, quando muoviamo il mouse in avanti (verso il monitor), il puntatore sale verso l'alto sullo schermo; questa associazione non è naturale, ma è ormai standardizzata e facilmente automatizzata dagli utenti.

- **Hard control** → **Soft control**: muovo il joystick e sposto un cursore virtuale.
- **Soft control** → **View**: spostando il cursore, cambio la visuale della scena.



Cambiare una di queste **trasformazioni spaziali** può disorientare l'utente, che deve reimparare il comportamento: come è successo nell'inversione verticale nei trackpad Apple: coerente internamente, ma diversa da ciò che molti utenti si aspettano.

Il **guadagno controller-display** (CD Gain) è il rapporto tra lo spostamento fisico del controller e lo spostamento visivo sul display: se muovo il mouse di 3 cm e il puntatore si muove di 3 cm (guadagno = 1), se il puntatore si muove di 6 cm (guadagno = 2). Il



guadagno può essere: costante o variabile in funzione della velocità. 77

2.9 IL MODELLO CONCETTUALE

Il **modello concettuale** è una rappresentazione mentale che l'utente si crea per comprendere come funziona un sistema. Per progettare un'interfaccia efficace, è fondamentale che il design rispecchi questo modello mentale, facilitando così l'uso del sistema.

Un buon modello concettuale si costruisce facendo leva su esperienze pregresse dell'utente, tramite metafore e congruenze fisiche. Alcuni esempi sono:

- Controller up → View up (muovo la visuale verso l'alto), Controller up → Content up (muovo il contenuto verso l'alto)
- Negli anni '70-'80, con la nascita delle prime interfacce grafiche, si è deciso di costruire l'esperienza utente partendo da ambienti familiari come l'ufficio e non da 0; facendo

nascere così metafore come: Desktop, Cartelle, Cestino, Archivi.

2.10 LE MODALITÀ

Una **modalità**, nell'HCI, è una configurazione funzionale in cui un'interfaccia assume comportamenti diversi a seconda dello stato attivo. Questo implica che gli stessi controlli (tasti, menu, gesti) possono avere effetti differenti a seconda della **modalità selezionata**. Per esempio:

- Il tasto Caps Lock attiva una modalità che trasforma tutte le lettere digitate da minuscole a maiuscole.
- Il tasto F9 in Microsoft Word può avere funzioni diverse a seconda della modalità attiva (aggiornamento campi, attivazione layout, ...).

L'uso delle modalità può semplificare l'interfaccia, ma va gestito con attenzione: se non è chiaro in quale modalità si trova il sistema, l'utente può fare errori. È quindi fondamentale **fornire feedback visivo o sonoro** chiaro che comunichi all'utente la modalità attiva, per evitare ambiguità.

78

2.11 ERRORI DI INTERAZIONE

Quando progettiamo un'interfaccia ciò che davvero conta sono le performance dell'utente:

L'utente è riuscito a completare il compito?

La tecnologia lo ha aiutato o ostacolato?

Per rispondere, è utile analizzare gli **errori di interazione**, ovvero quei casi in cui il sistema fallisce nel supportare efficacemente l'utente.

Vediamo quattro esempi ordinati dal più grave (ma spesso evitato) al meno grave (ma molto comune):

DELUDERE LE ASPETTATIVE

Le interfacce devono rispettare le **aspettative mentali** dell'utente. Un messaggio come:

“Salvare i cambiamenti?” → Sì

“Annullare i cambiamenti?” → No



può sembrare corretto, ma se l'utente è abituato alla prima formulazione, cambiare improvvisamente la domanda può portare a clic errati e perdita di dati. Non basta adattare la risposta alla domanda: bisogna anche considerare **quanto l'utente si aspetta una certa forma**.

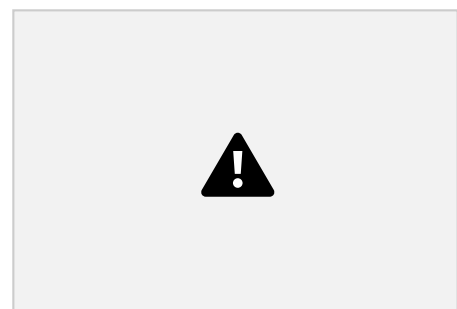
NON DARE SUGGERIMENTI

Un esempio classico è la password inserita con il Caps Lock attivo. Se il sistema non avvisa l'utente, quest'ultimo continuerà a digitare la password corretta in modo errato, rischiando il blocco dell'account.

Un **piccolo messaggio di avviso** può prevenire frustrazione e errori evitabili. 79

ESSERE TROPPO VELOCI

Durante operazioni come il trascinamento del testo, lo scroll automatico può risultare troppo rapido. Questo comportamento porta spesso l'utente **lontano dalla posizione desiderata**, costringendolo a interrompere e ricominciare.



La soluzione? Riconoscere quando l'utente sta eseguendo un'azione precisa e **adattare dinamicamente la velocità** dello scorrimento.

AVANZARE O NON AVANZARE?

Un esempio frequente è l'inserimento di dati, come un codice numerico: il cursore avanzerà automaticamente ogni 3 cifre?

Qui non c'è un errore tecnico, ma una **mancanza di indicazioni chiare**. L'interfaccia **non comunica** cosa succederà, lasciando l'utente nell'incertezza.

